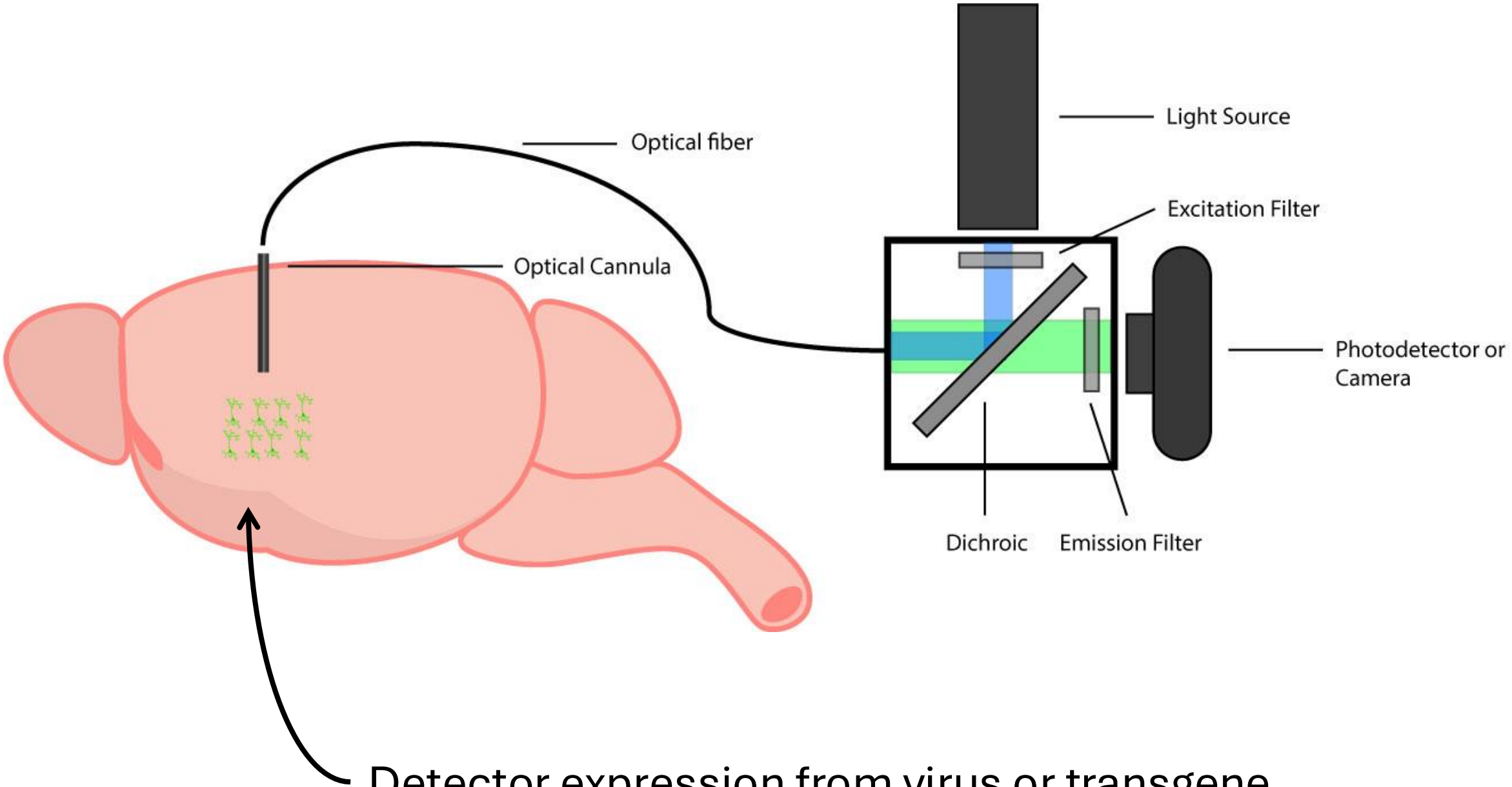
Fiber Photometry Data Processing in Python

Griffin Golde



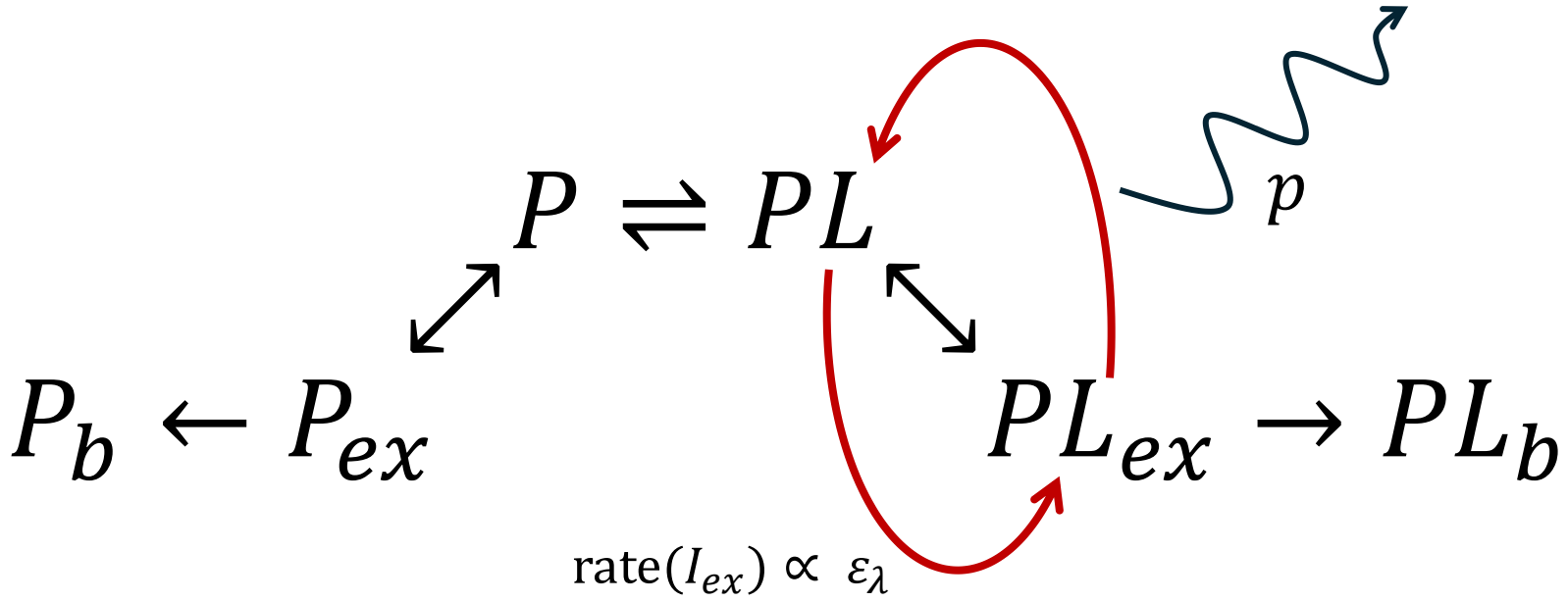
Credit: https://www.mightexbio.com/fiber-photometry/

$$F(\lambda) = f(\theta)g(\lambda) \cdot \Phi_F I_{ex} \cdot \varepsilon_\lambda b\boldsymbol{c}$$

$f(\theta)$ = collection efficiency
$g(\lambda)$ = quantum yield of detector

Φ_F = quantum yield of fluorophore
I_{ex} = excitation wavelength intensity

ε_λ = molar extinction coefficient
b = optical path length
$\boldsymbol{c}$ = fluorophore concentration



P = protein unbound
PL = protein-ligand complex

X_{ex} = excited state
X_b = bleached state

Experimental λ:

$$\varepsilon_{PL} \gg \varepsilon_P$$

Isosbestic λ:

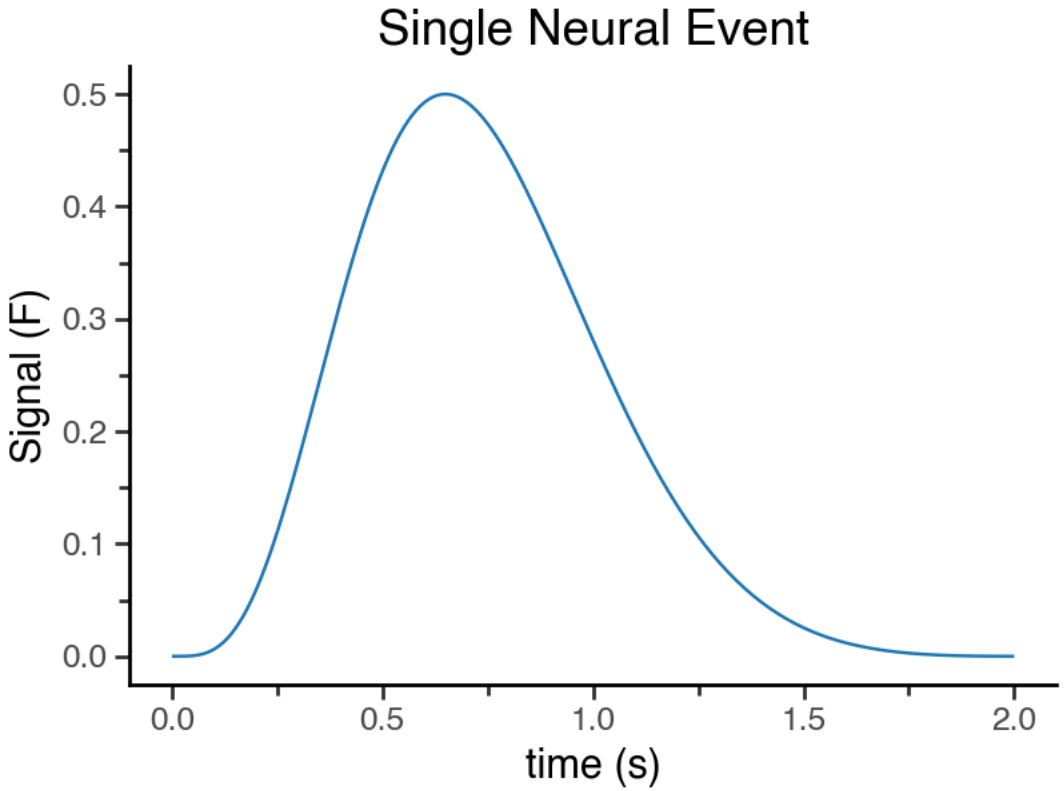
$$\varepsilon_{PL} = \varepsilon_P$$

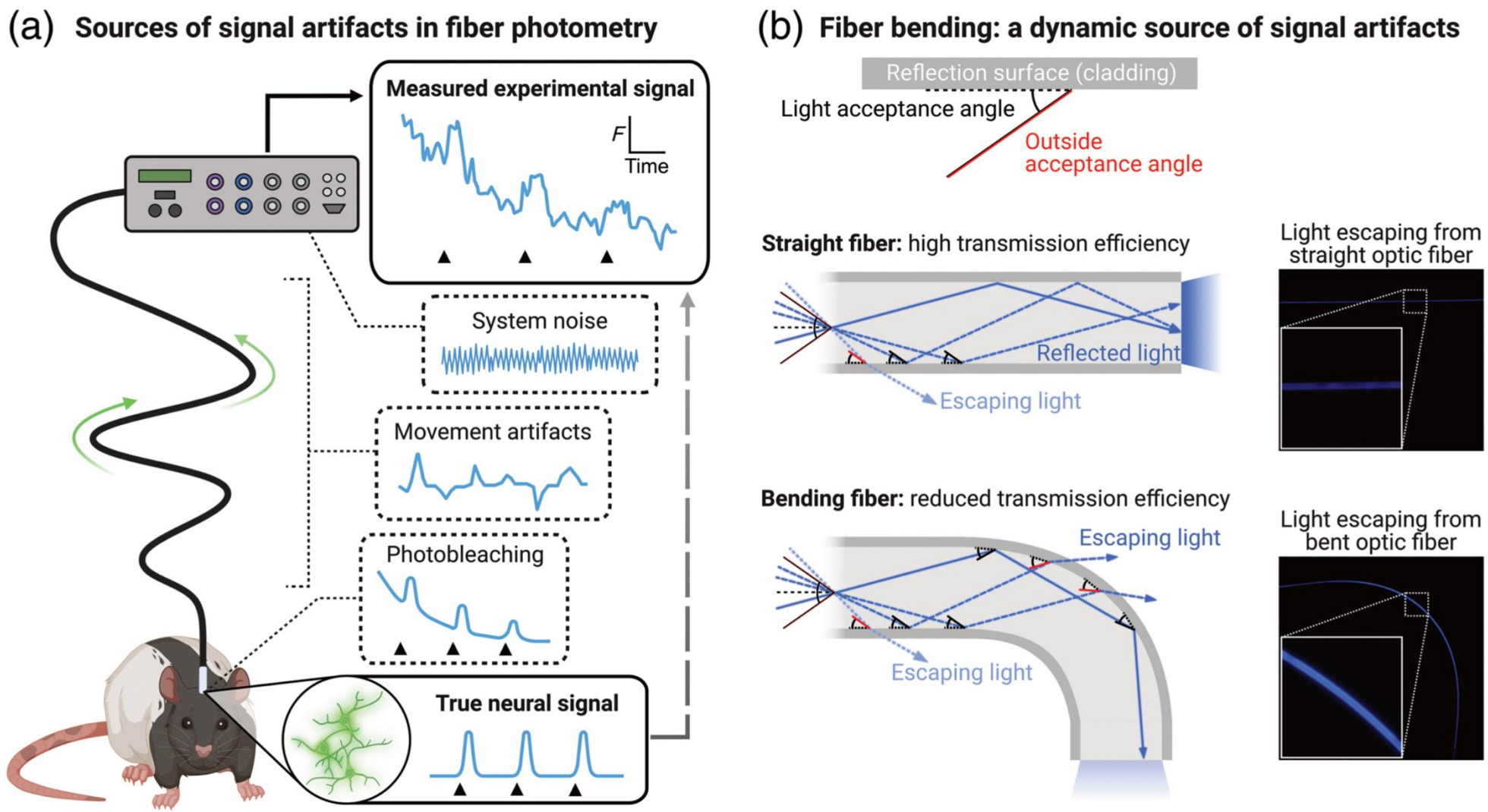
$F(\lambda)$ at the isosbestic point is agnostic to [L]

Elements of photometry data

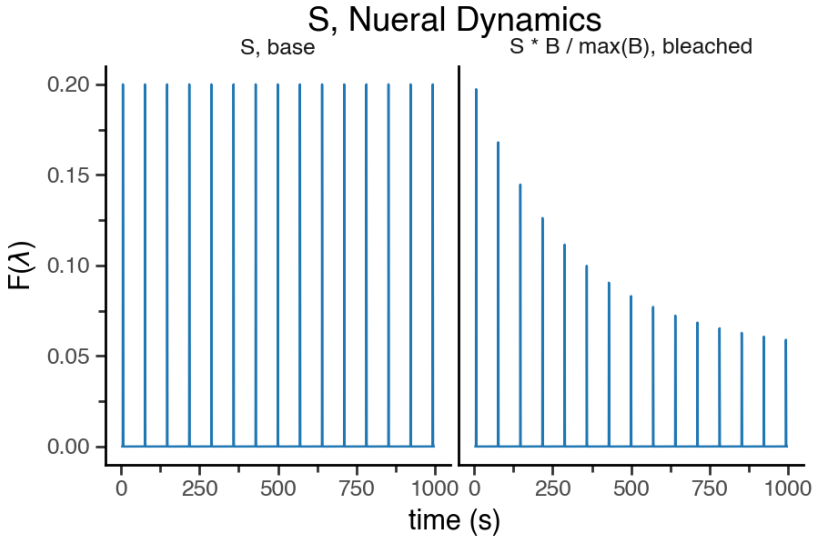
1. Neural dynamics (S)
2. Photobleaching attenuation (B)
3. Movement / hemodynamic artifacts (M)
4. Photonic noise (σ_{dep})
5. Detector / electronic noise (σ_{ind})
6. Background autofluorescence

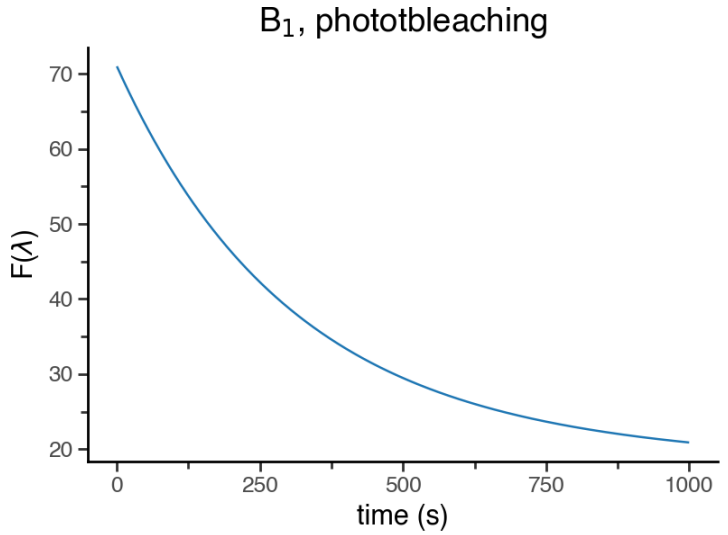
Goal: retrieve neural dynamics (S)

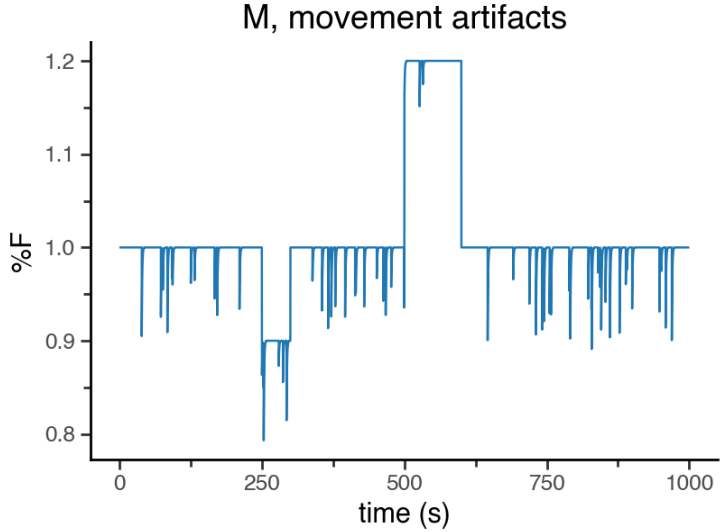


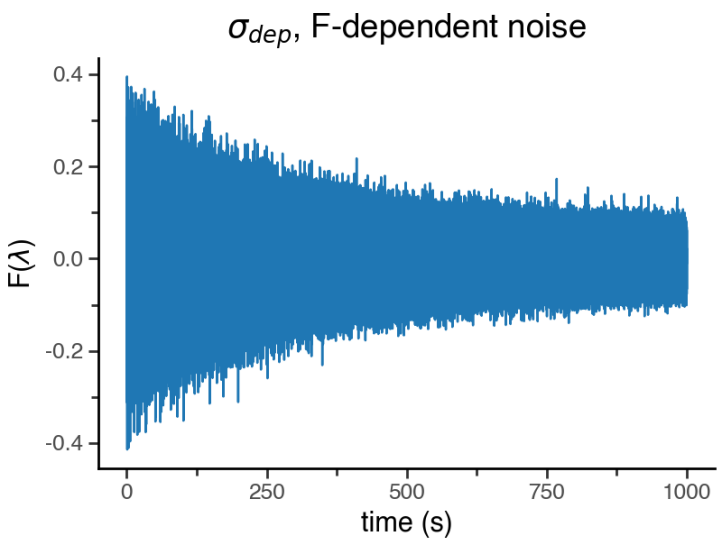


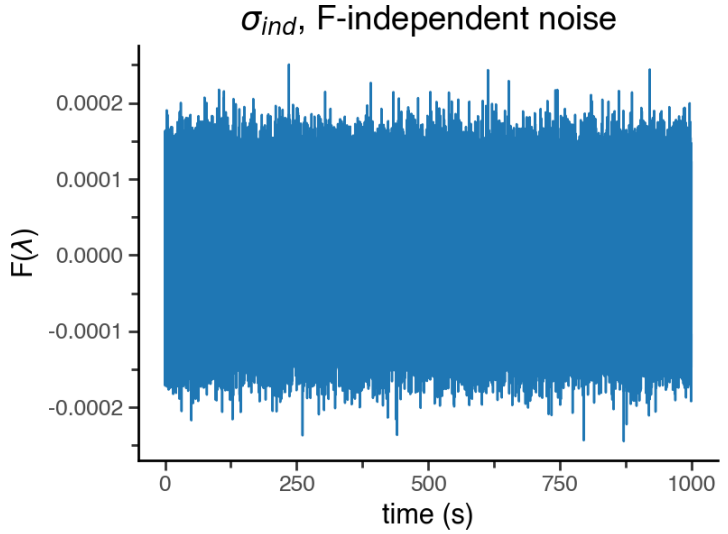
Credit: Jean-Richard-dit-Bressel and Keevers [1]

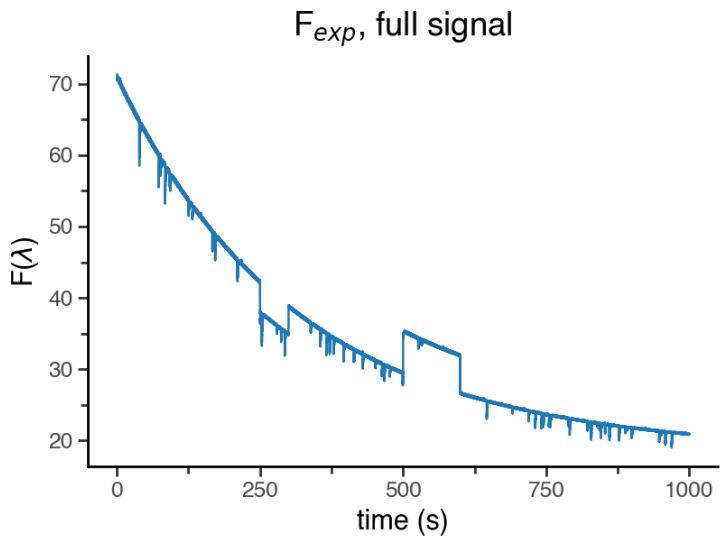




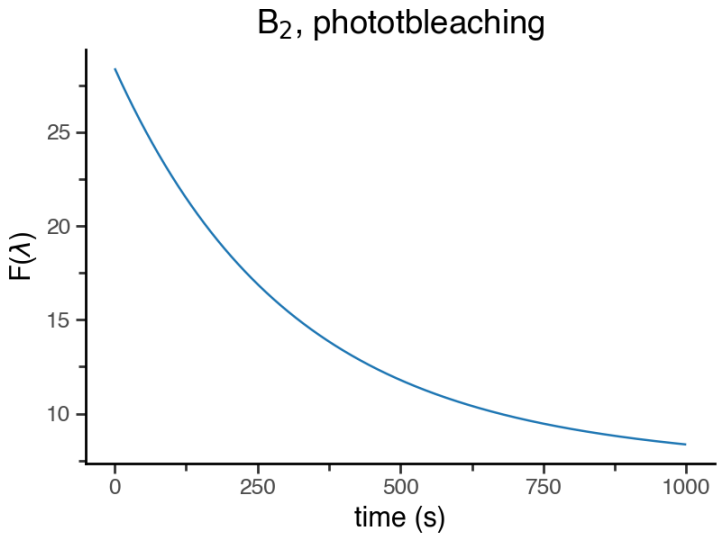


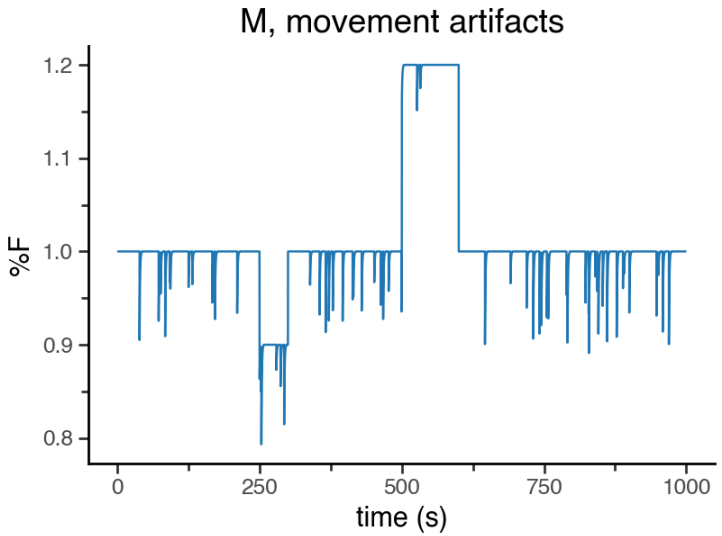


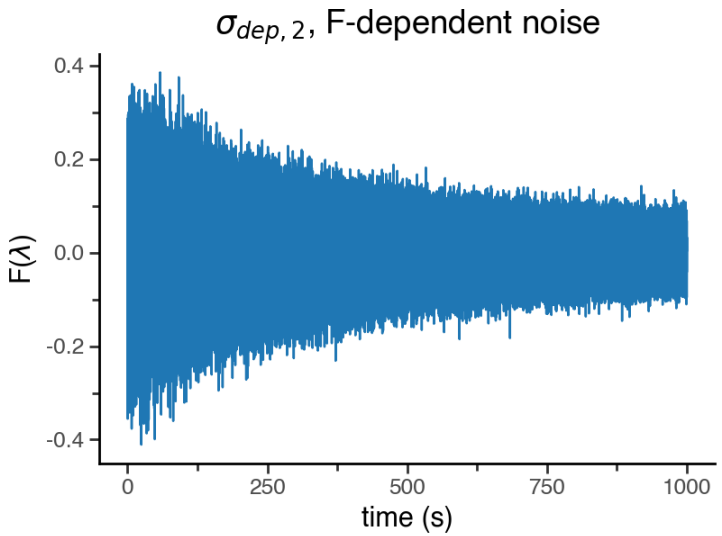


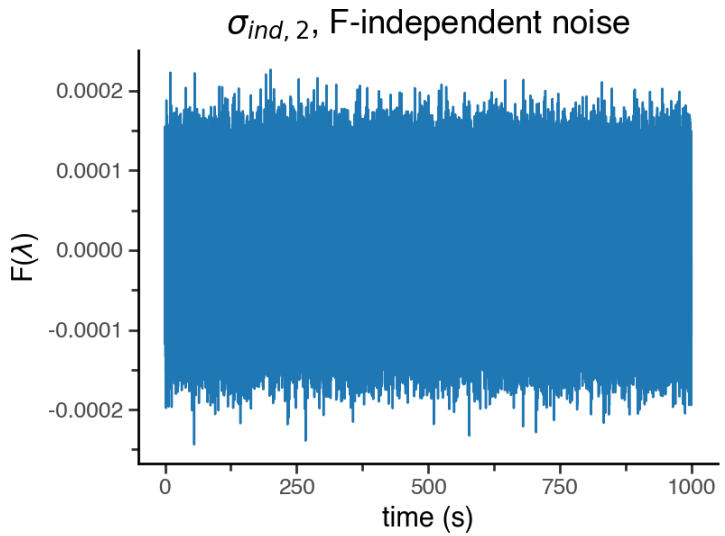


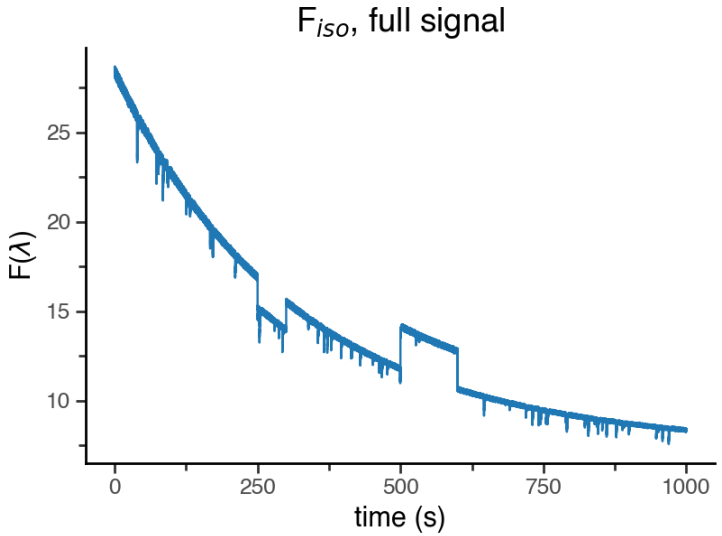
$$F_{exp} = \left(S \frac{B}{\max(B)} + B \right) \times M + \sigma_{ind} + \sigma_{dep}$$











$$F_{iso} = B_2 \times M + \sigma_{ind,2} + \sigma_{dep,2}$$

$$F_{exp} \text{ vs. } F_{iso}$$

$$S \neq 0$$

$$M = M$$

$$B_1 \approx \beta_0 + \beta_1 B_2$$

$$|\sigma_{ind,1}| = |\sigma_{ind,2}|$$

$$|\sigma_{dep,1}| = \beta_1 |\sigma_{dep,2}|$$

$$*|\sigma_{dep}| \gg |\sigma_{ind}|$$

Processing with isosbestic

1. Preprocessing

- Low pass filtering and downsampling

2. Fit isosbestic to experimental

- Usually with least-square minimization on:

$$\hat{F}_{exp} = \beta_0 + \beta_1 \cdot F_{iso}$$

3. Perform isosbestic correction

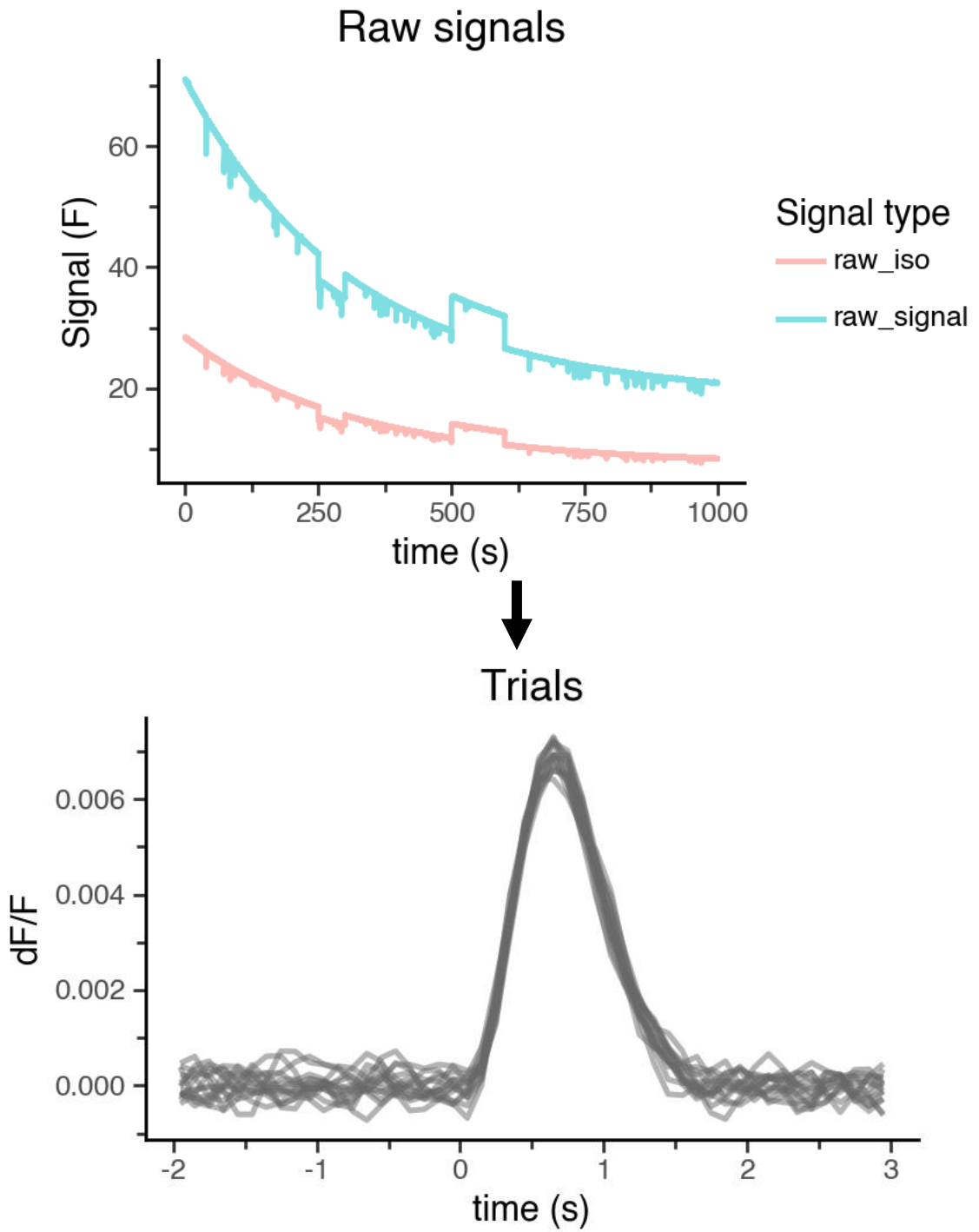
- dF or dF/F to correct ***M***

4. Window data

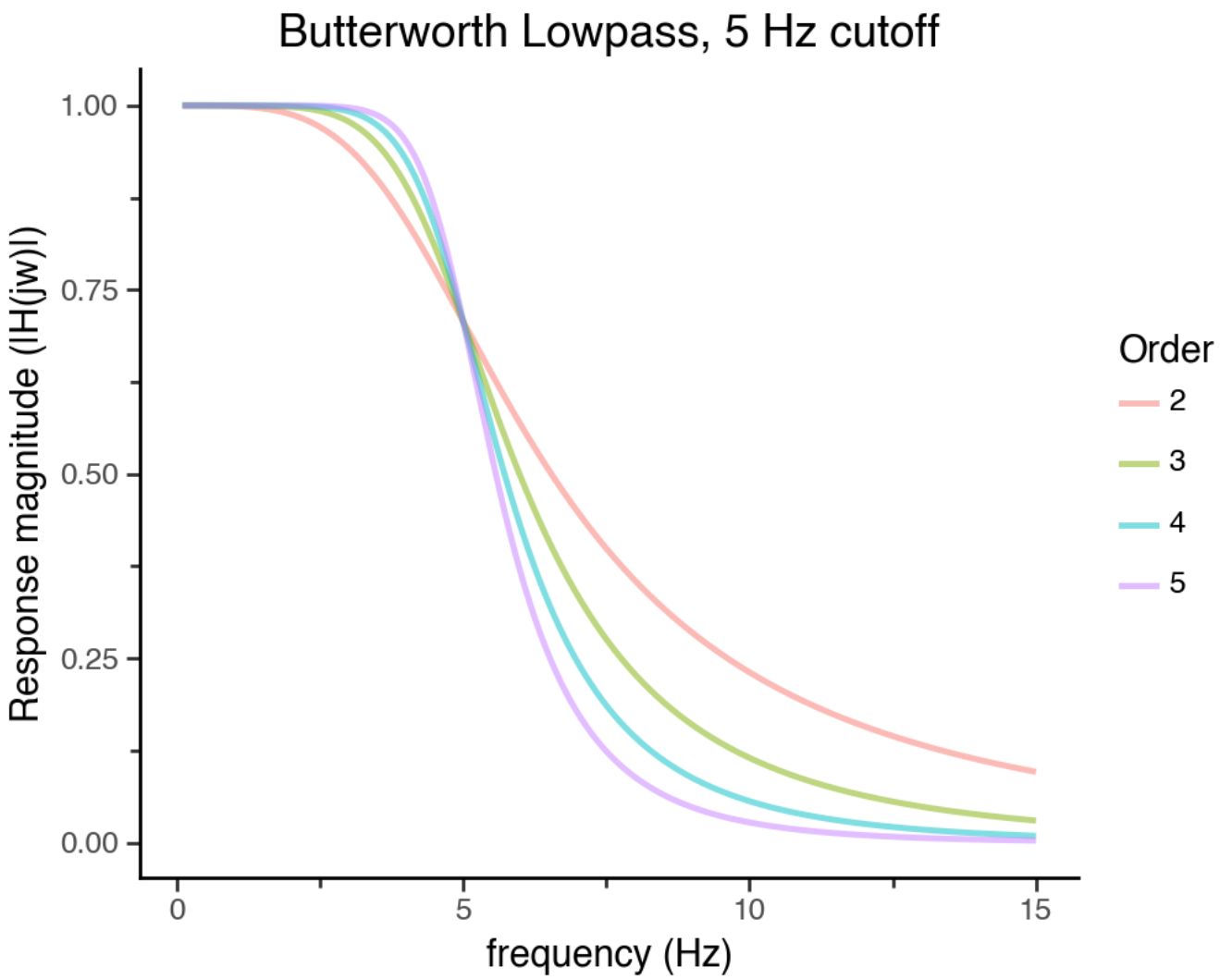
- 1D to 2D transformation while preserving events

5. Perform normalization (optional)

- Additional trial-wise normalizations



1. Preprocessing



$$H(j\omega) = \frac{1}{\sqrt{1 + \left(\frac{\omega}{\omega_c}\right)^{2n}}}$$

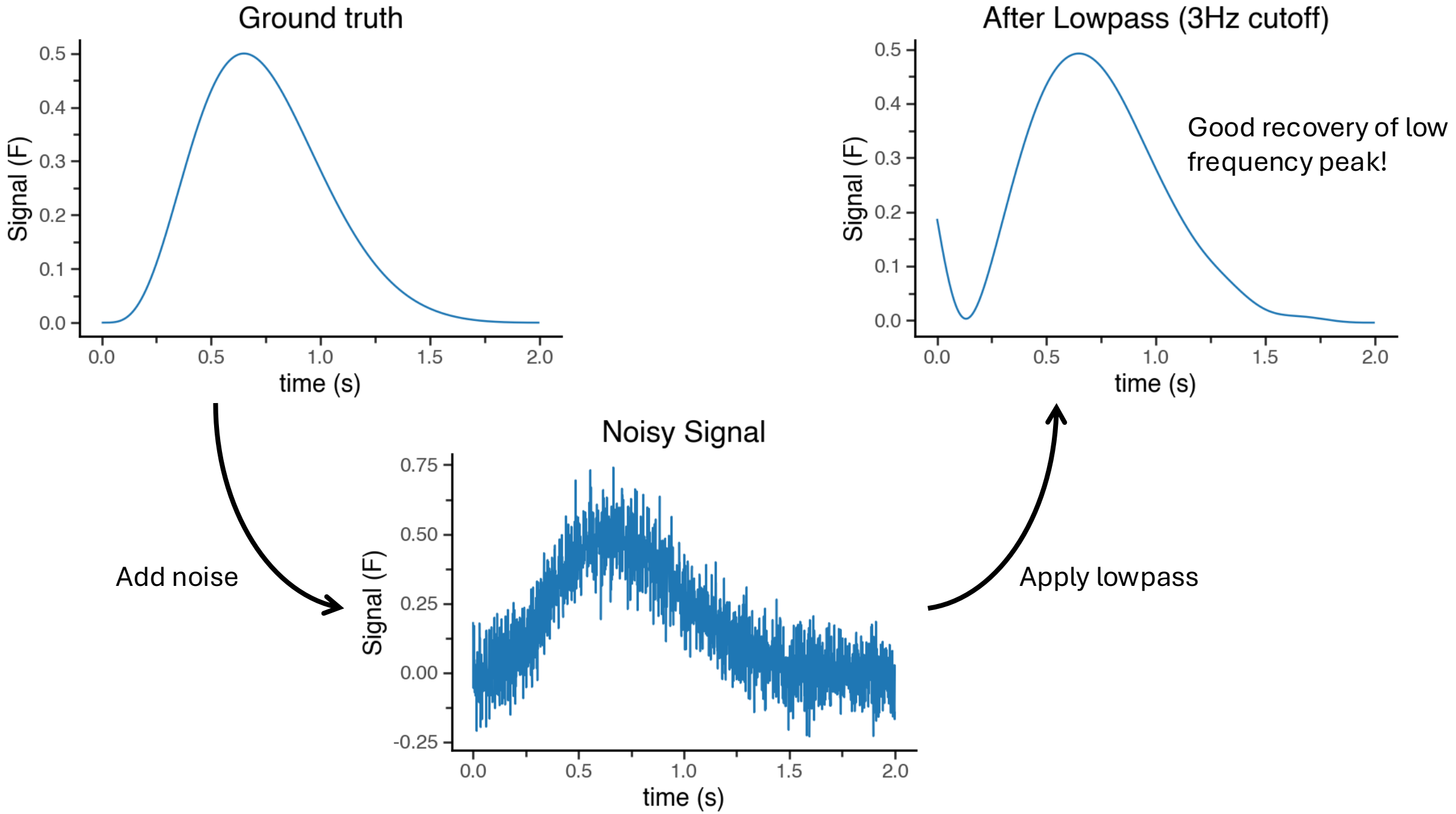
ω = frequency

ω_c = cutoff frequency

n = order

- Lowpass filtering preserves low frequencies while filtering high ones
- Good for photometry since:

$$\omega_{\text{of interest}} \ll \omega_{\text{sampling}}$$


Ground truth
Signal (F)
time (s)
After Lowpass (3Hz cutoff)
Signal (F)
time (s)
Good recovery of low frequency peak!
Noisy Signal
Signal (F)
time (s)
Add noise
Apply lowpass

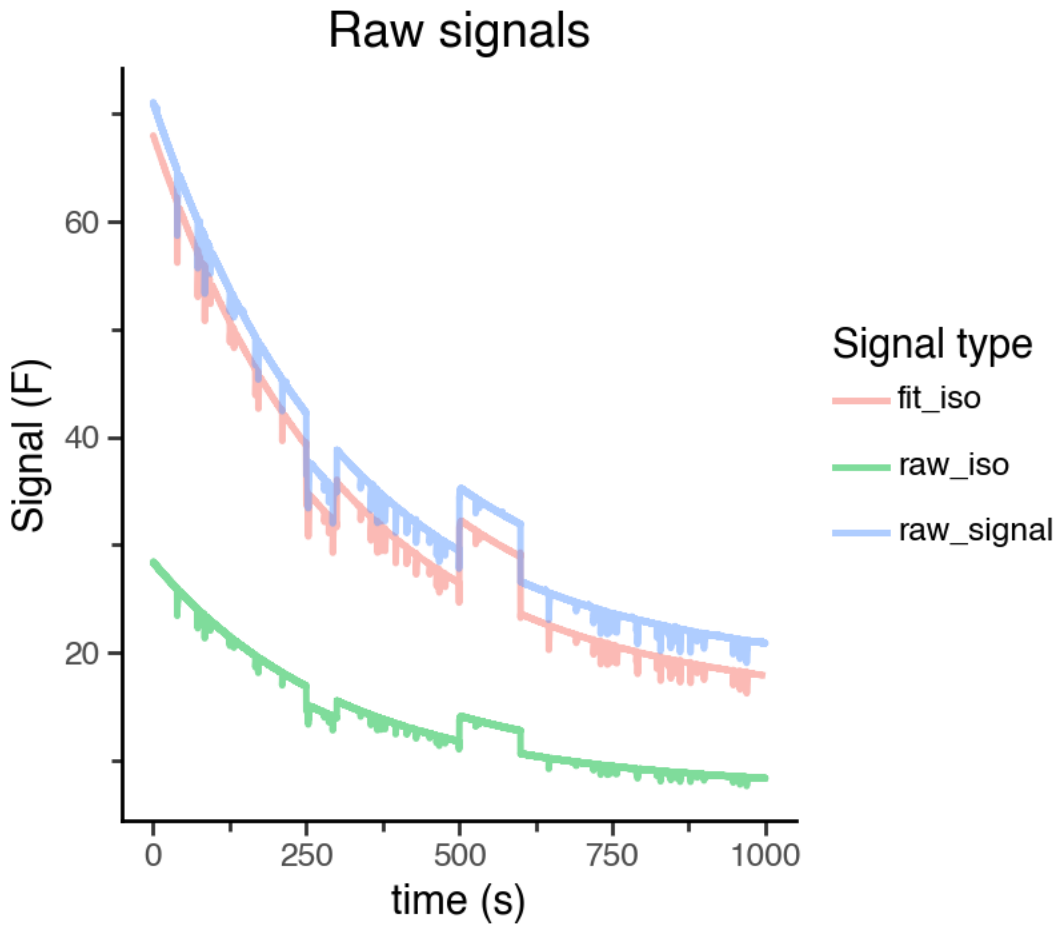
2. Fit isosbestic to experimental

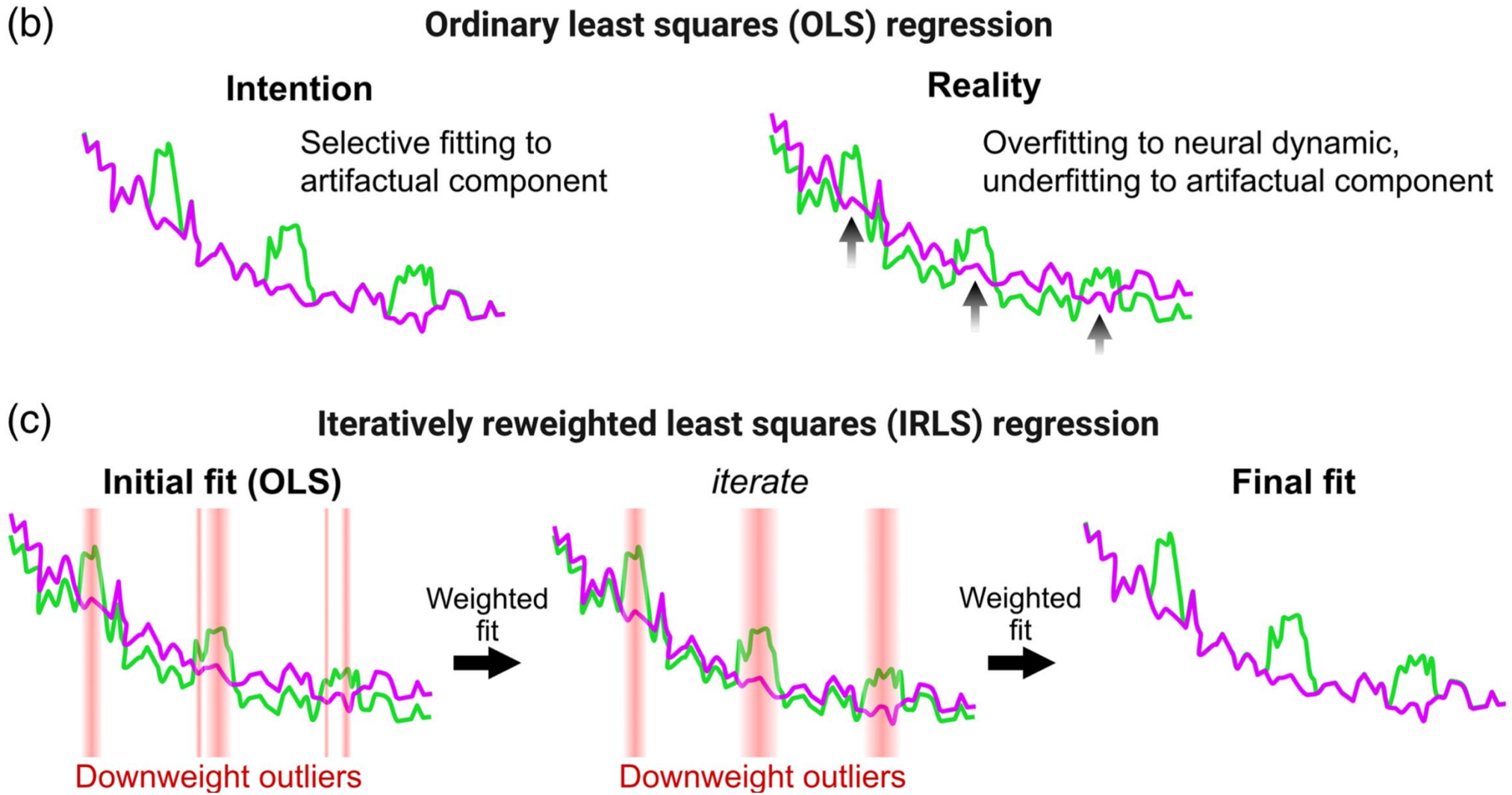
$$\hat{F}_{exp} = \beta_0 + \beta_1 \cdot F_{iso}$$

$$\hat{F}_{exp} = \beta_1 \cdot F_{iso}$$

Usually with:

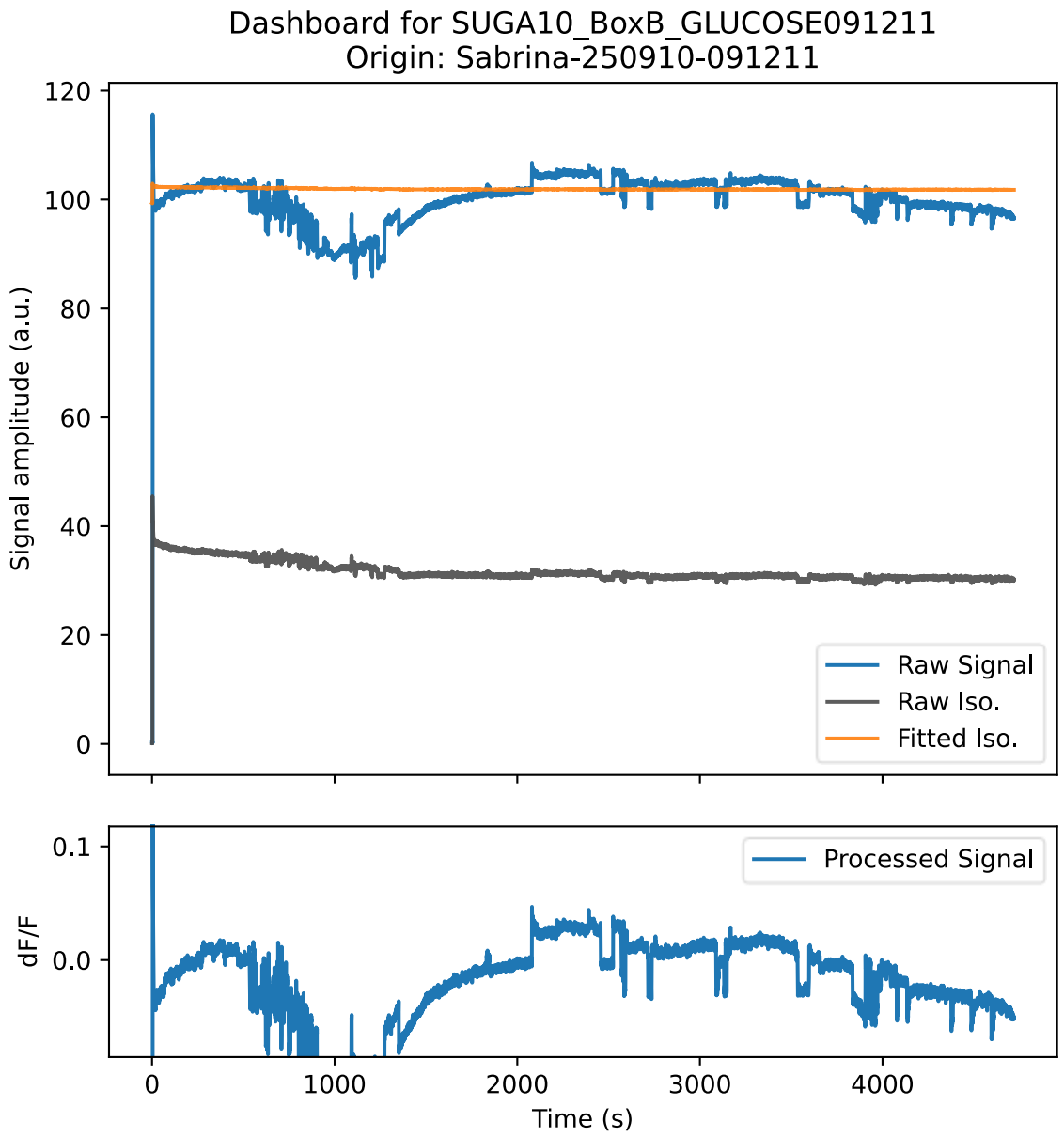
- Ordinary-Least Squares (OLS)
- Iteratively-Reweighted (IRLS)



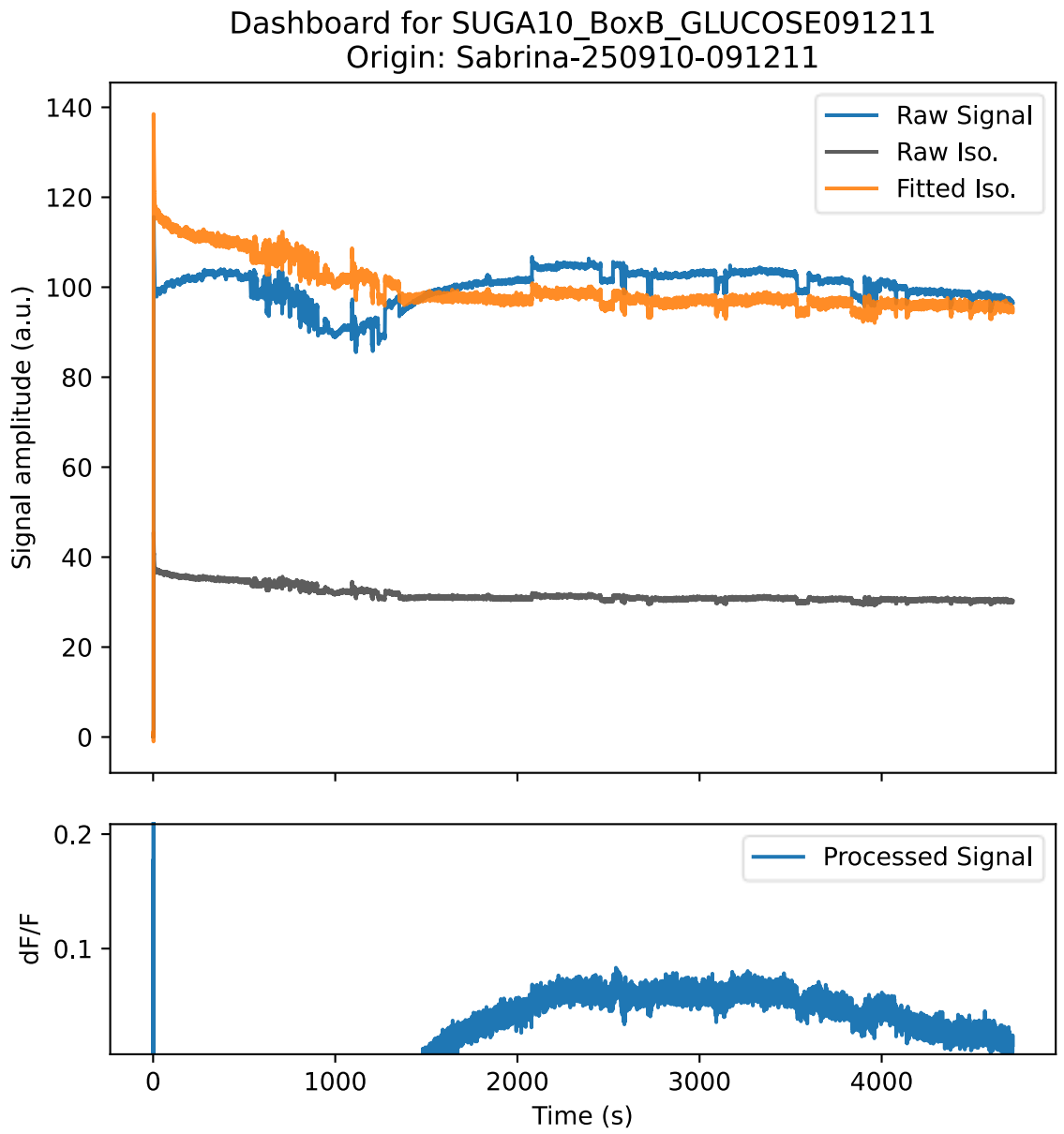


Credit: Jean-Richard-dit-Bressel and Keevers [1]

IRLS



IRLS, no intercept

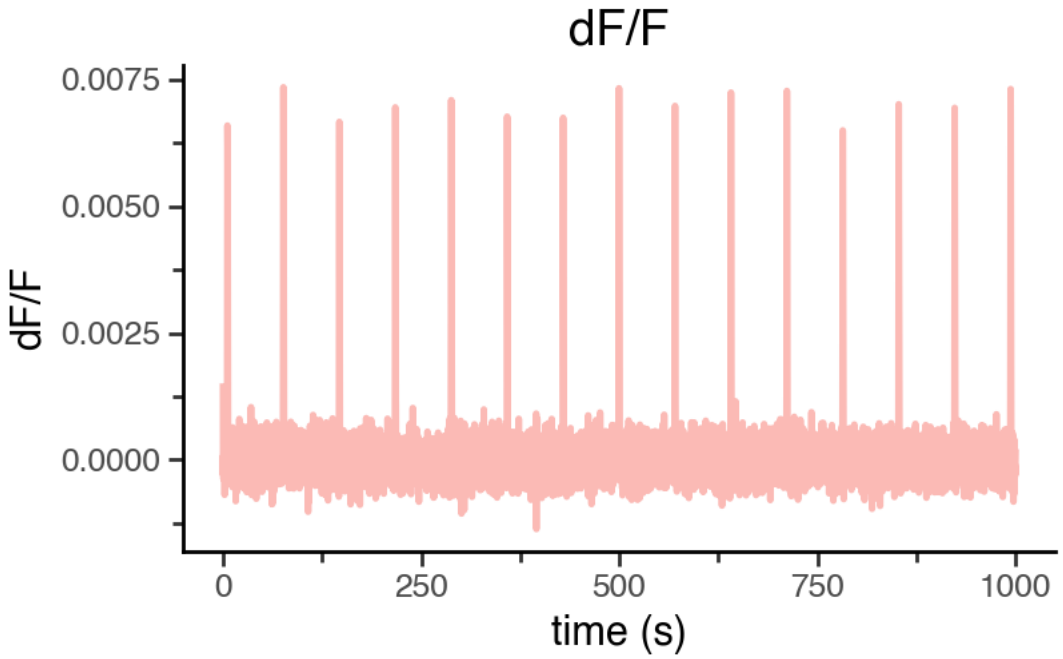


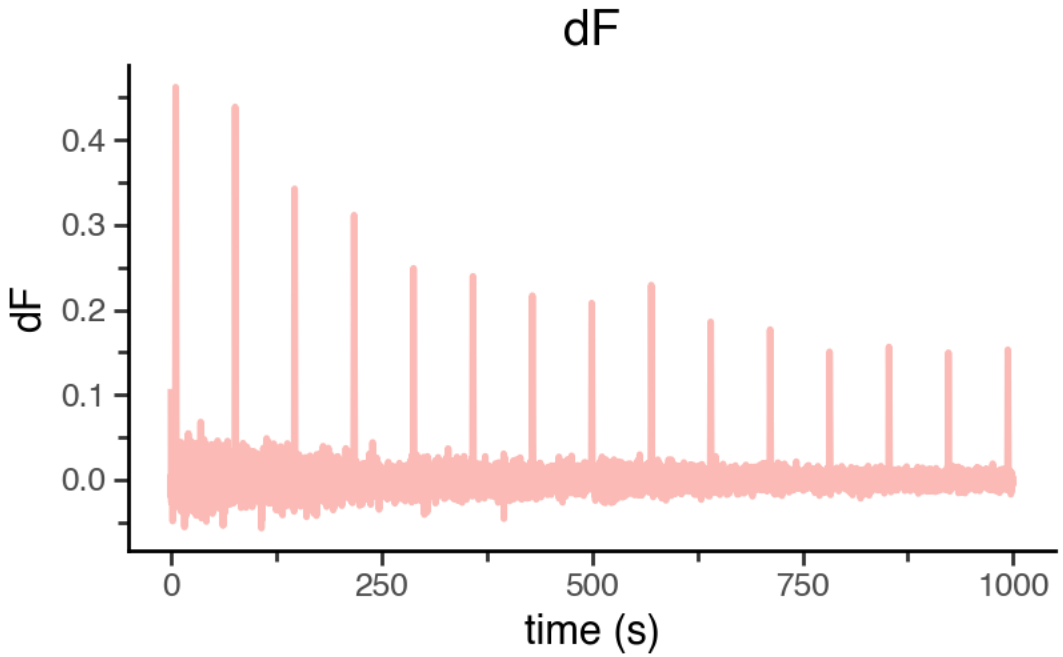
3. Isosbestic correction

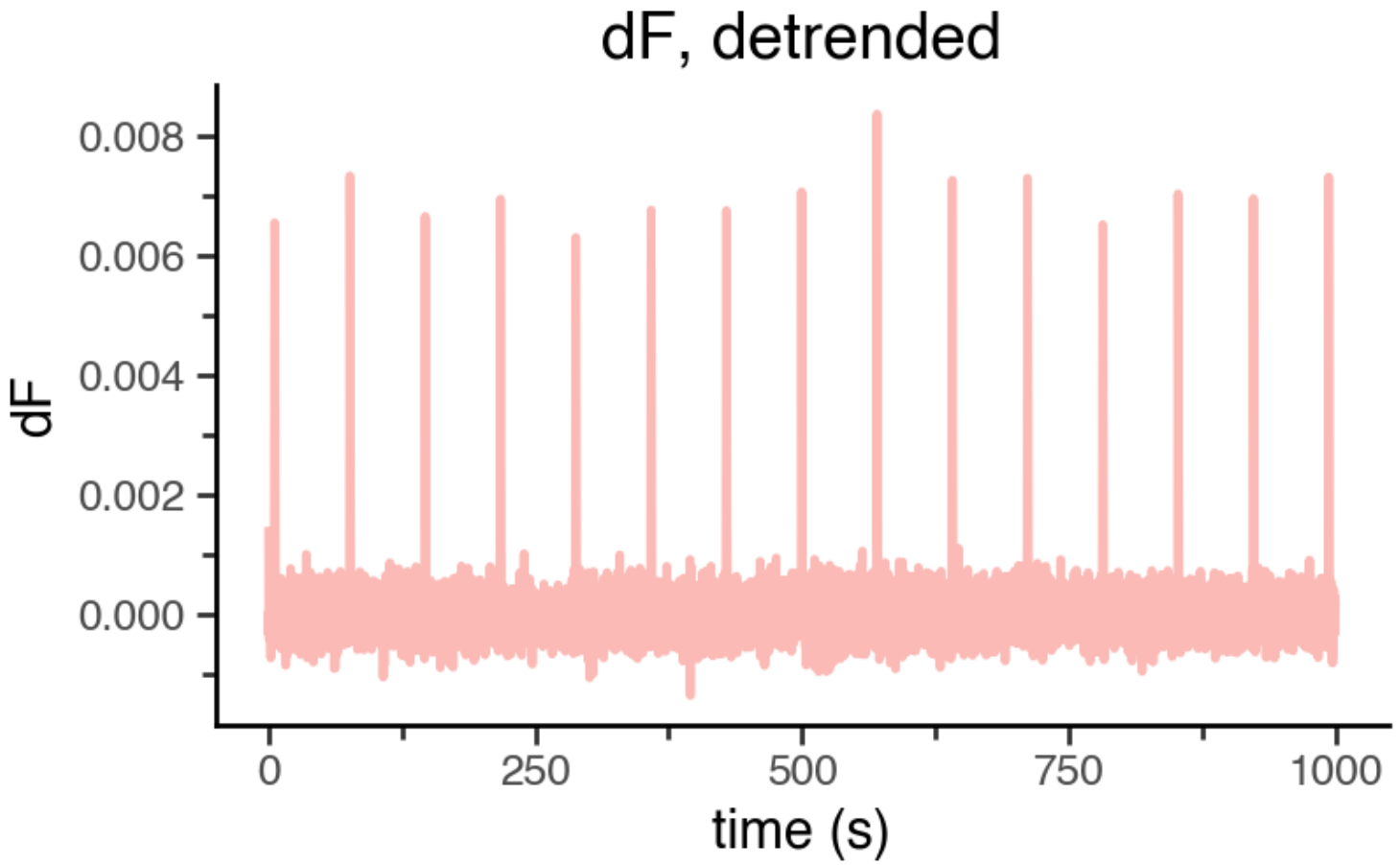
$$\frac{dF}{F} = \frac{F_{exp} - \hat{F}_{exp}}{\hat{F}_{exp}}$$

$$dF = F_{exp} - \hat{F}_{exp}$$

**dF* does not correct for bleaching attenuation on its own







$$\hat{B}_{sig} = -\alpha_1 e^{-\beta_1} - \alpha_2 e^{-\beta_2} + c$$

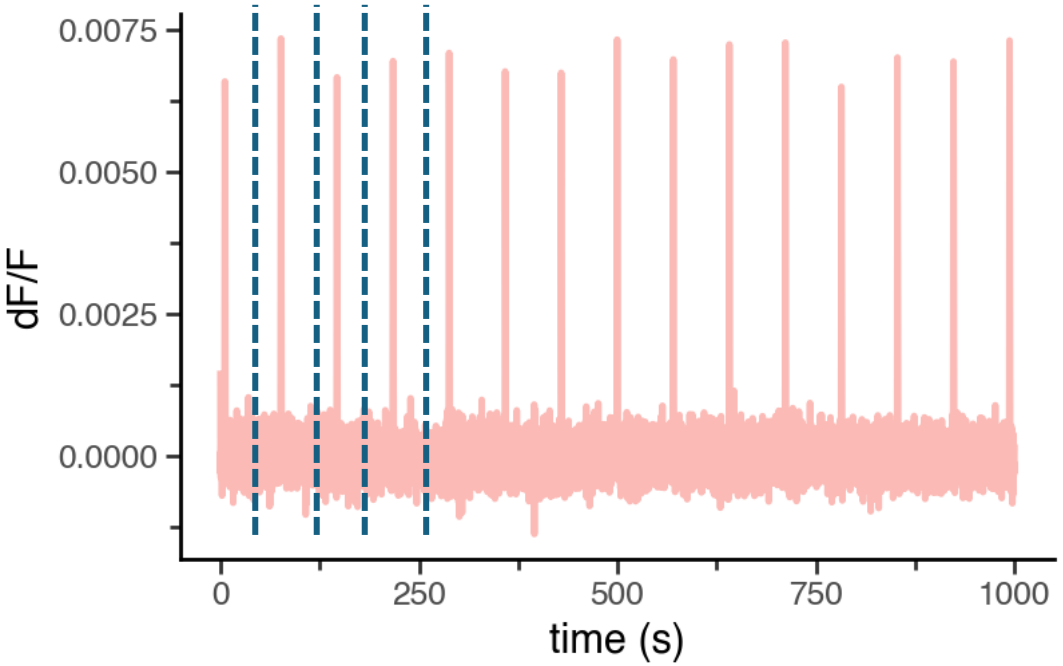
$$D_{sig} = \frac{F_{sig} - \hat{B}_{sig}}{\hat{B}_{sig}}$$

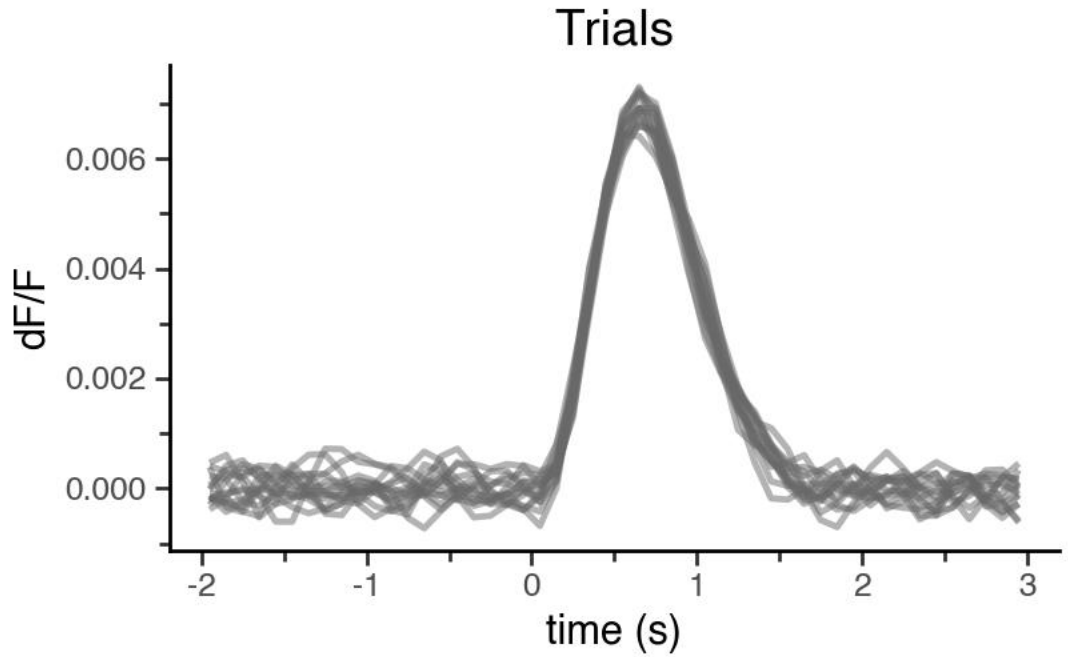
$$\text{“}dF\text{”} = D_{exp} - \hat{D}_{exp}$$

Why does it correct for bleaching attenuation?

It’s a dF/F!

4. Window data





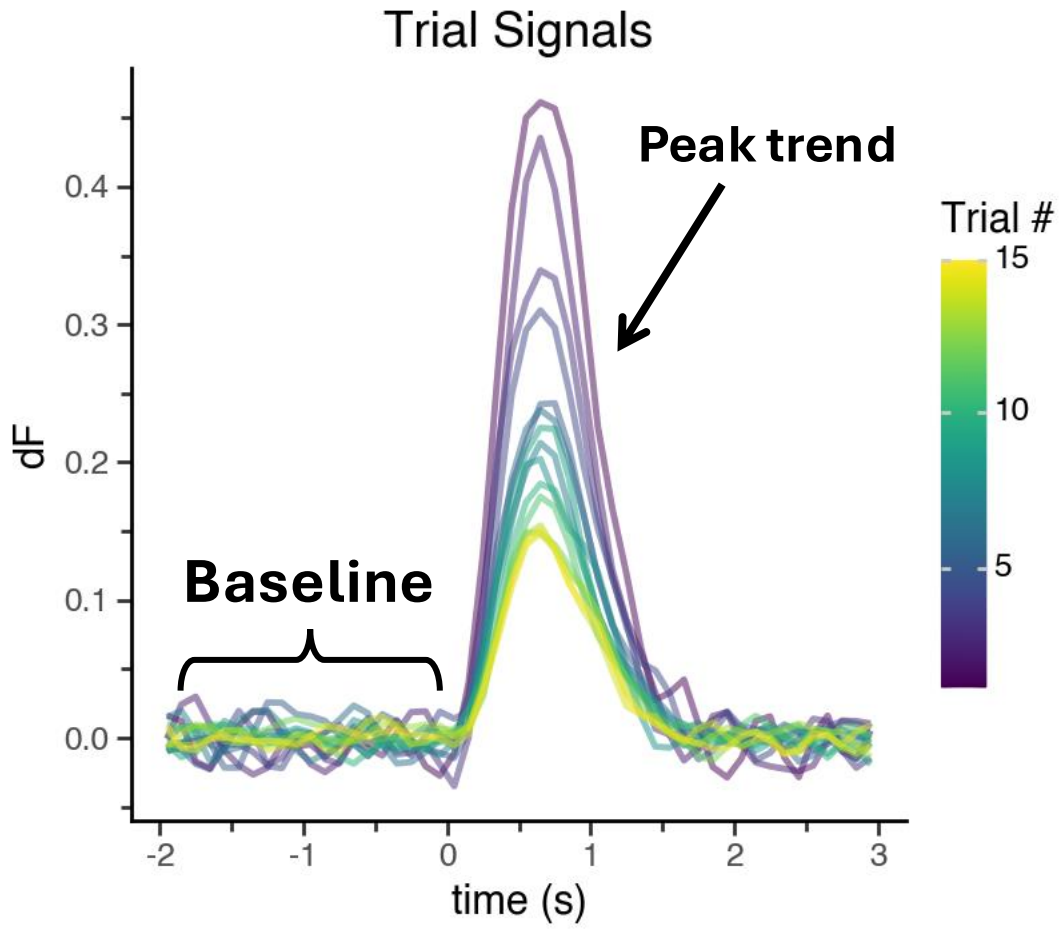
	Hsl	Lrg	Sml	Zap	trial_label	rat	current	date
0	-6.935183	NaN	0.0	NaN	Sml	NAc404	50	11/25/2024
1	-7.575470	0.0	NaN	NaN	UnPun	NAc404	50	11/25/2024
2	-5.687706	0.0	NaN	NaN	UnPun	NAc404	50	11/25/2024
3	-8.260157	NaN	0.0	NaN	Sml	NAc404	50	11/25/2024
4	-6.175949	0.0	NaN	NaN	UnPun	NAc404	50	11/25/2024

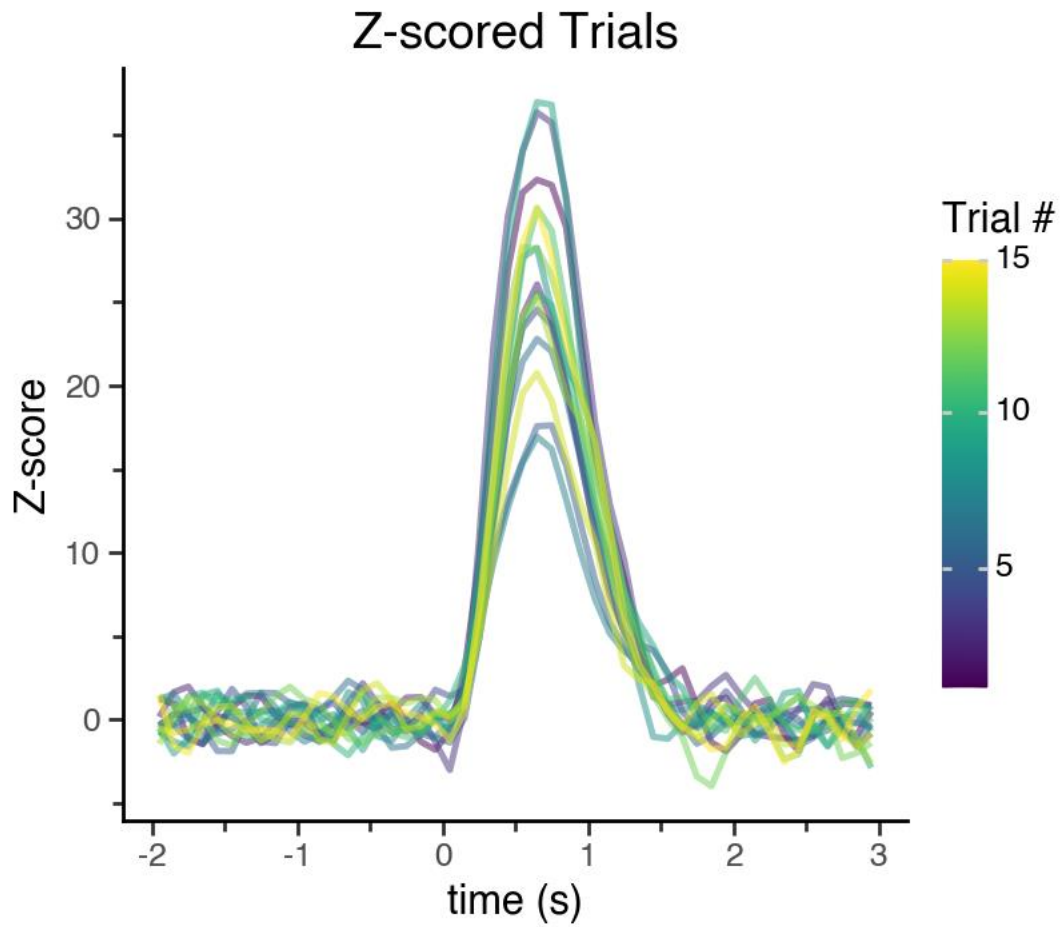
5. Trial-wise normalization (optional)

$$\textbf{zero} = F - \mathrm{mean}(baseline)$$

$$\textbf{Z-score} = \frac{F - \mathrm{mean}(baseline)}{\mathrm{std}(baseline)}$$

$$\textbf{Z-score} = \frac{dF - \text{mean}(baseline)}{\text{std}(baseline)}$$

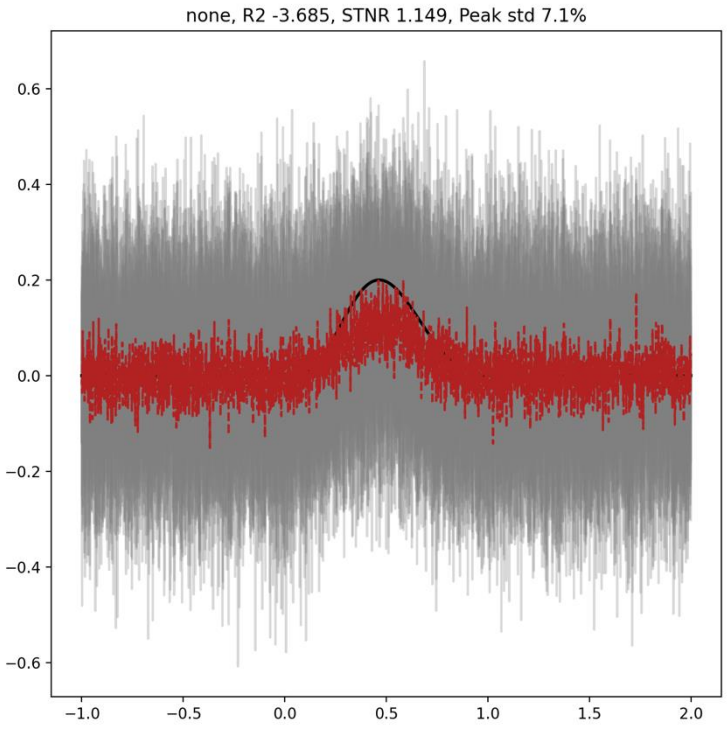




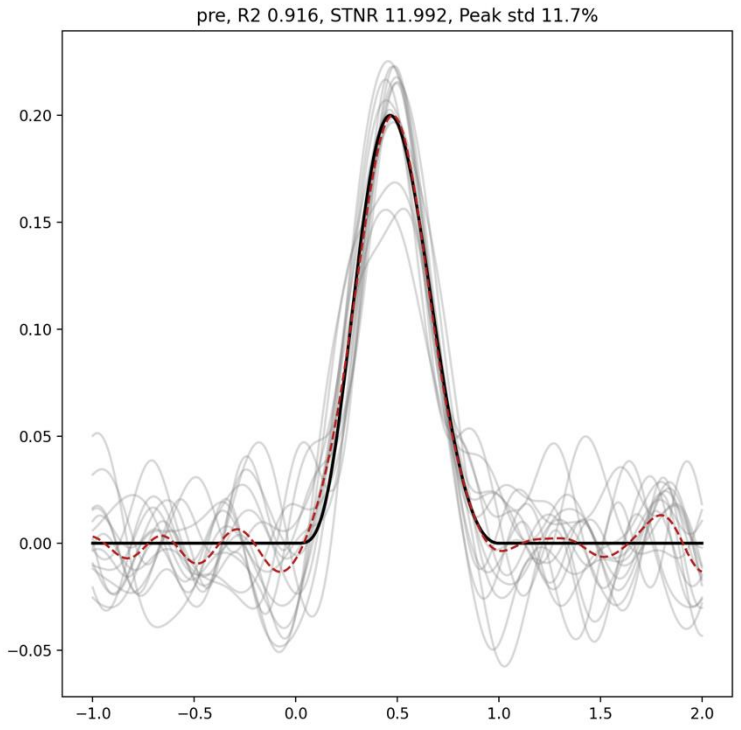
Whole-trial Z-scoring will not correct for photobleaching atteuation!

Method tests on simulated data

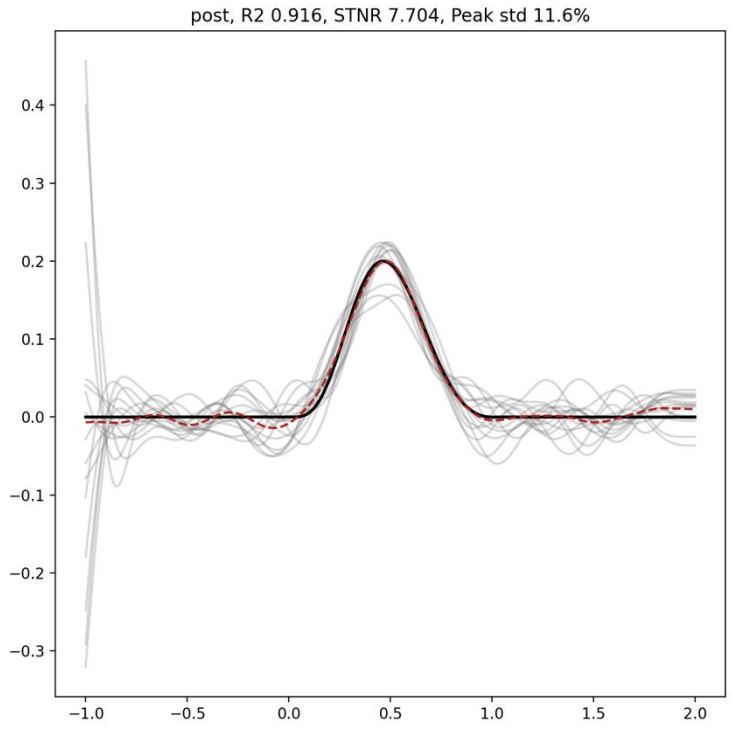
No Lowpass



Pre-correction



Post-correction



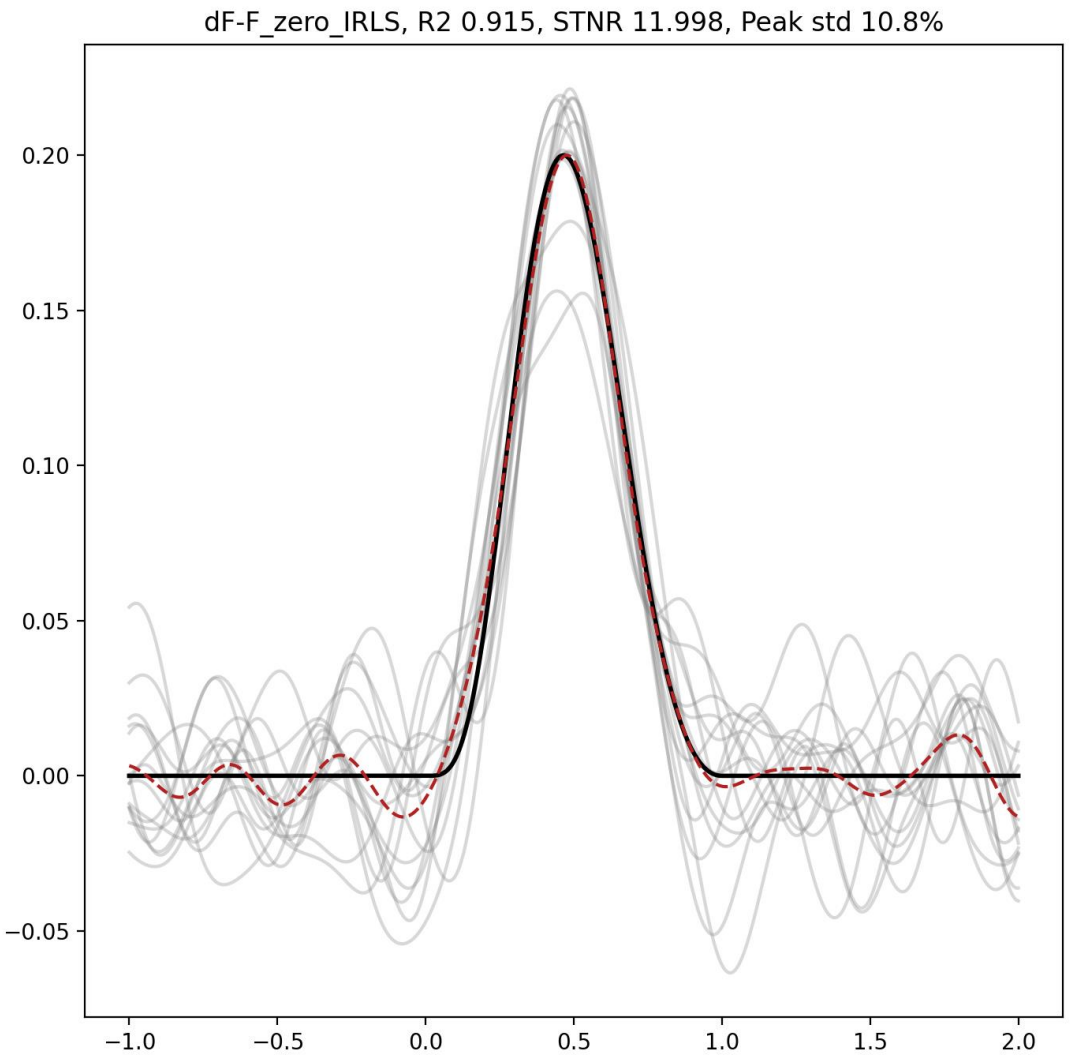
- Without a lowpass filter signal : noise ratios nosedive
- Lowpass pre and post isosbestic-correction

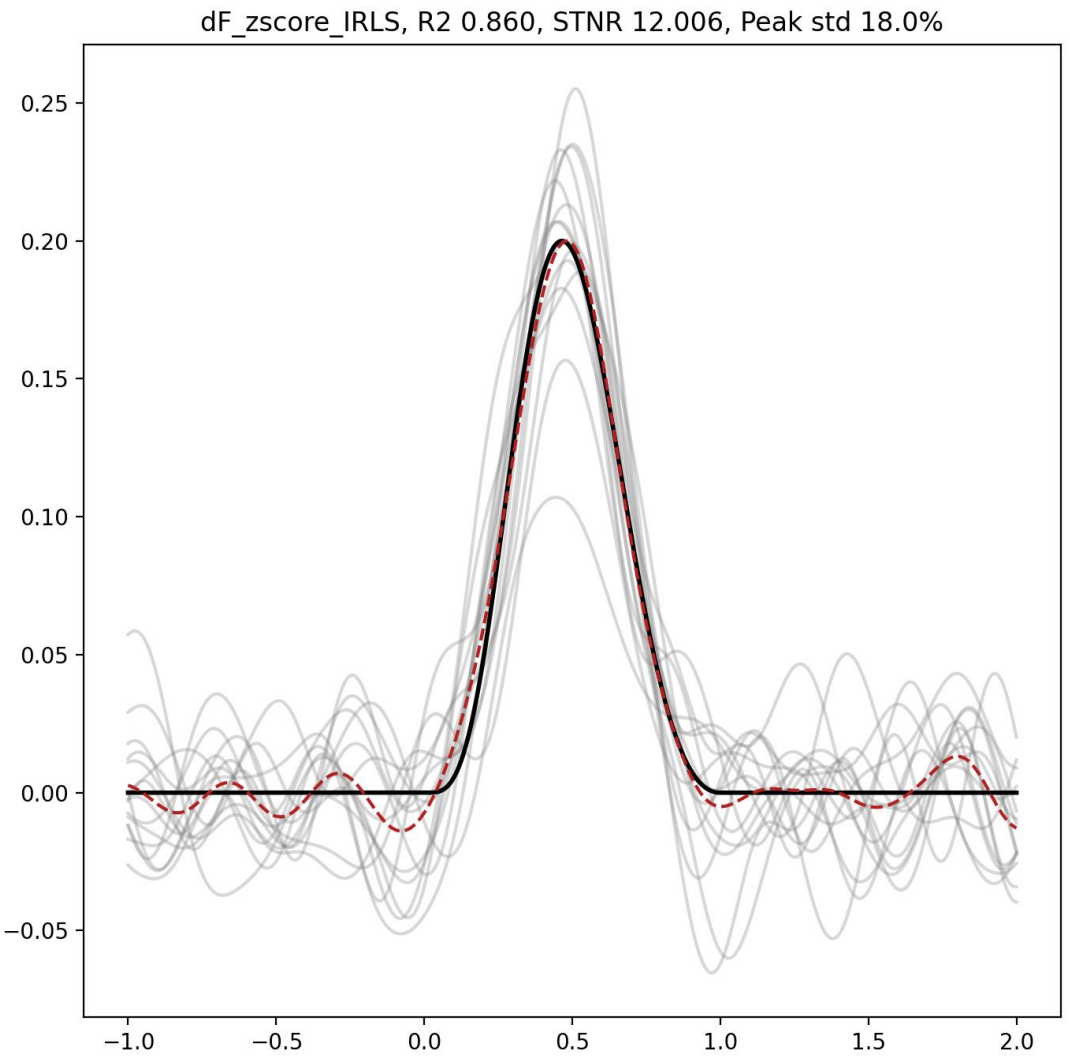
Hyperparameter grid search

Correction method	Normalization	Fit Method	R^2	Signal : noise	Peak std. (%)	Peak-time correlation	Correlation p-value
dF/F	none	IRLS	0.885	11.992	11.7%	0.032	0.91
		OLS	0.882	11.921	11.8%	0.068	0.81
	zero	IRLS	**0.884**	**11.998**	**10.8%**	**0.018**	**0.95**
		OLS	0.884	11.998	10.8%	0.018	0.95
	zscore	IRLS	0.861	11.998	17.0%	0.039	0.89
		OLS	0.861	11.998	17.0%	0.039	0.89
dF	none	IRLS	0.654	12.000	46.1%	-0.918	1.4E-06
		OLS	0.655	11.930	45.9%	-0.918	1.4E-06
	zero	IRLS	0.642	12.006	46.7%	-0.925	8.0E-07
		OLS	0.642	12.007	46.7%	-0.925	8.0E-07
	zscore	IRLS	**0.856**	**12.006**	**18.0%**	**-0.014**	**0.96**
		OLS	0.856	12.007	18.0%	-0.014	0.96

- No difference between IRLS and OLS
- dF without Z-score and dF/F with Z-score correction perform poorly
- dF/F-zero and dF-zscore perform similarly
- dF/F-zero has tighter distribution of recovered peaks

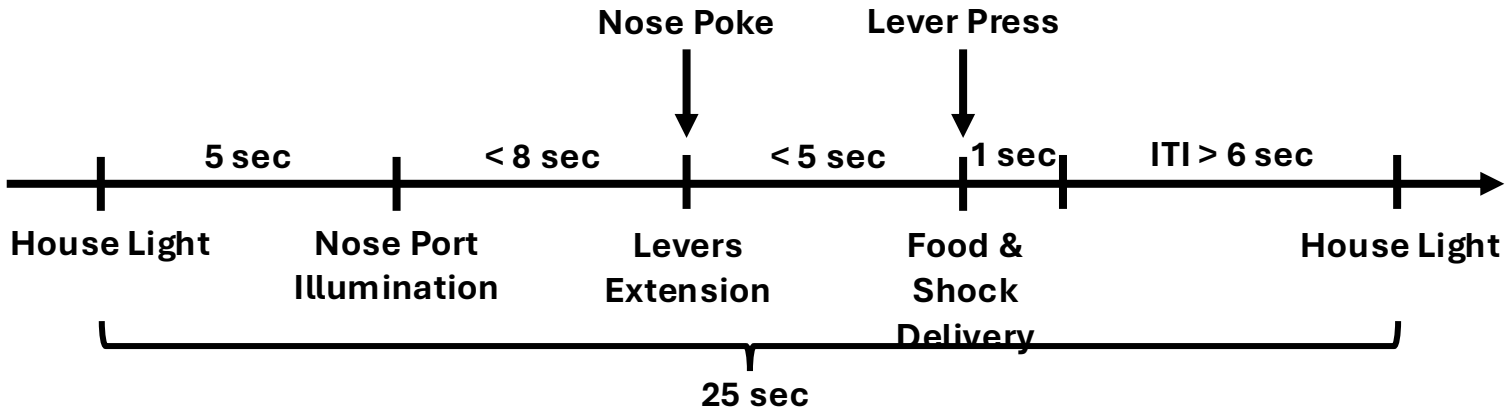
Two best methods: dF/F-zero and dF-zscore



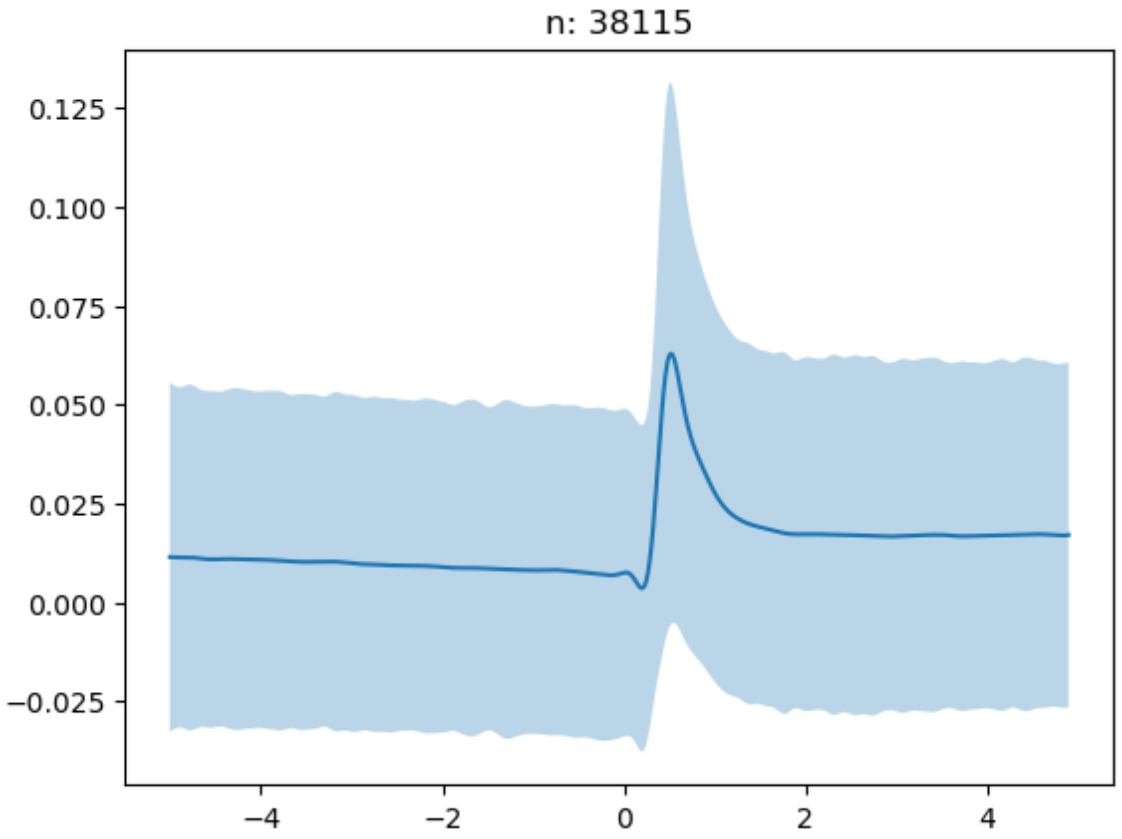


Comparisons with experimental data

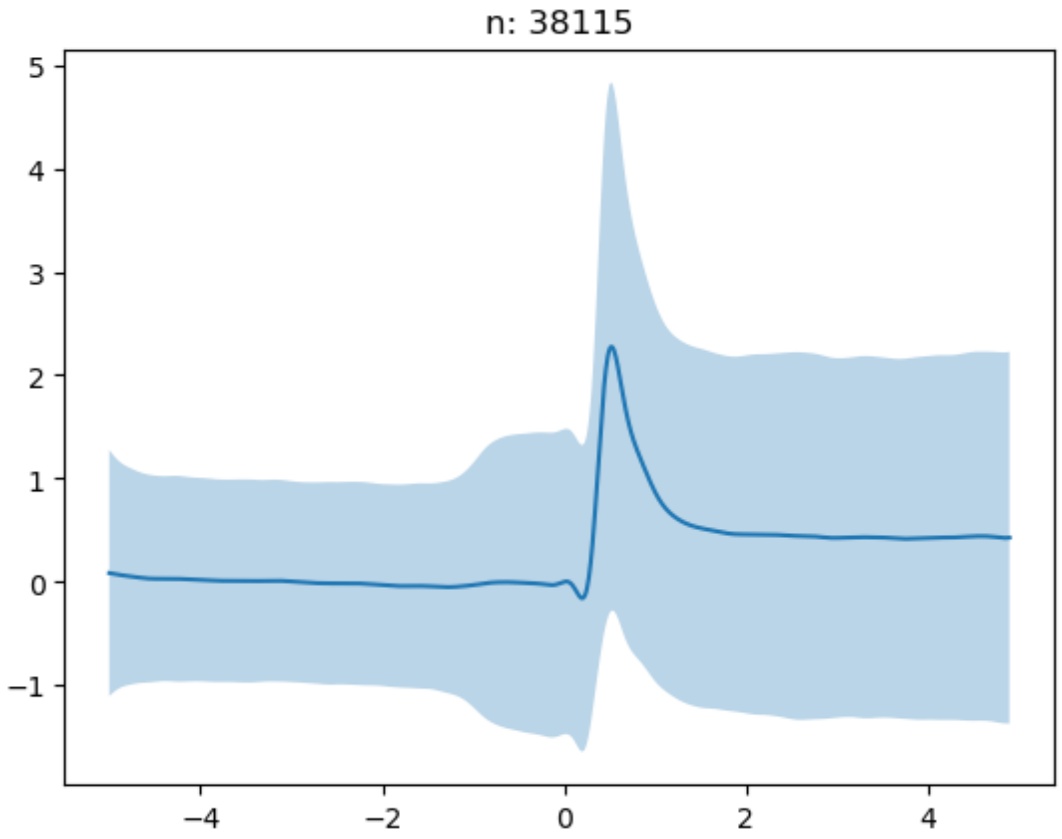
- Retrieving dopamine “houselight” peaks from a risky decision-making task (RDT)
- Data from 19 rats across age ~ sex cohorts
  - 5 AF, 5 YF, 5AM, 4YM

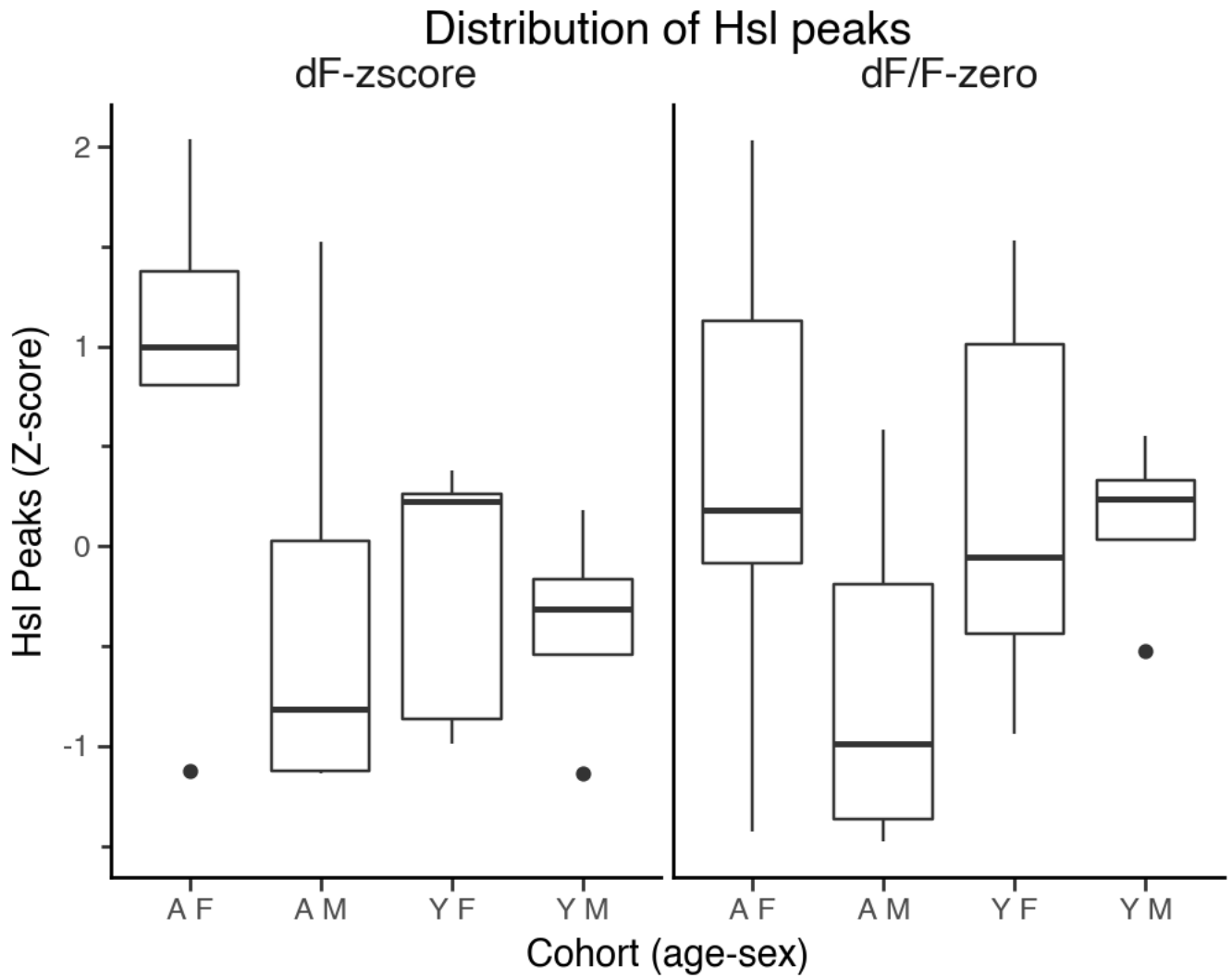


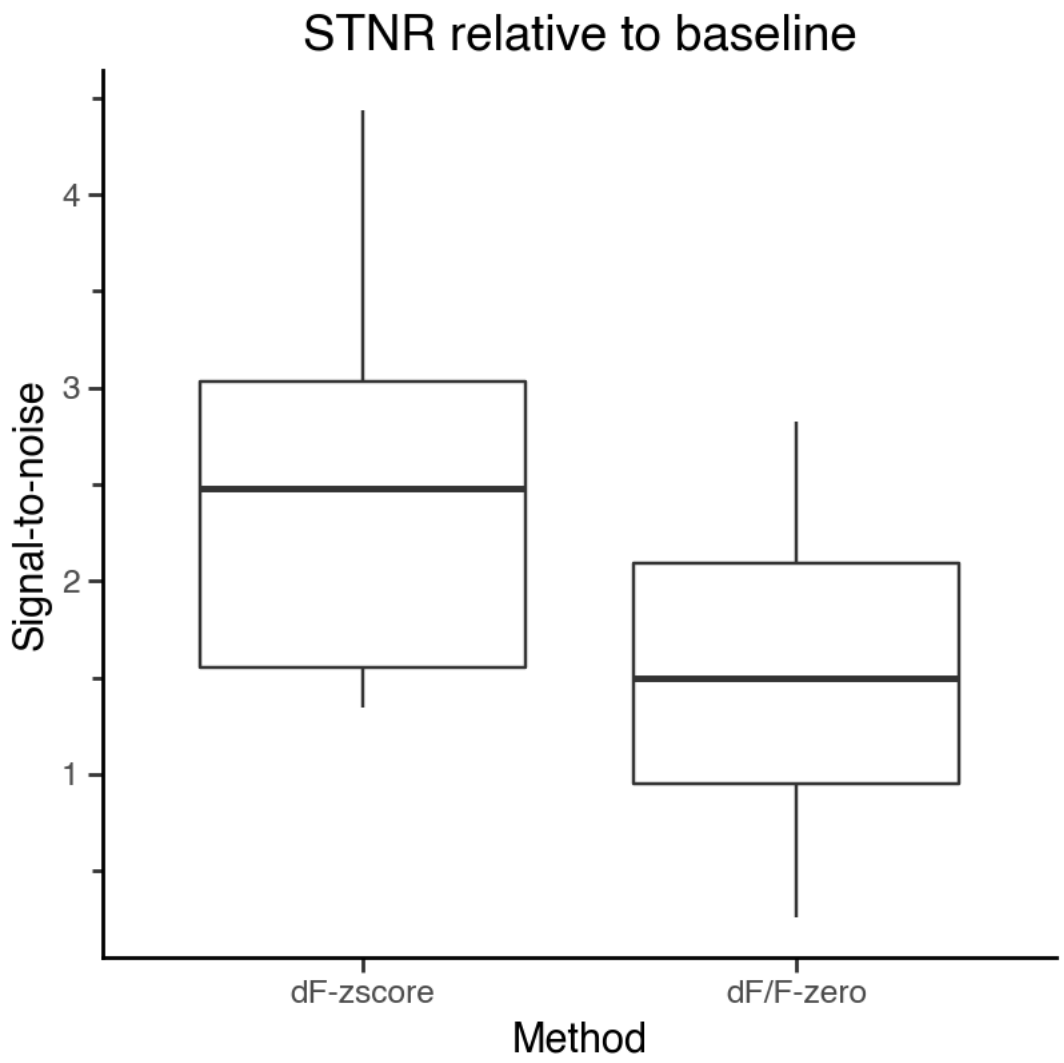
dF/F-zero

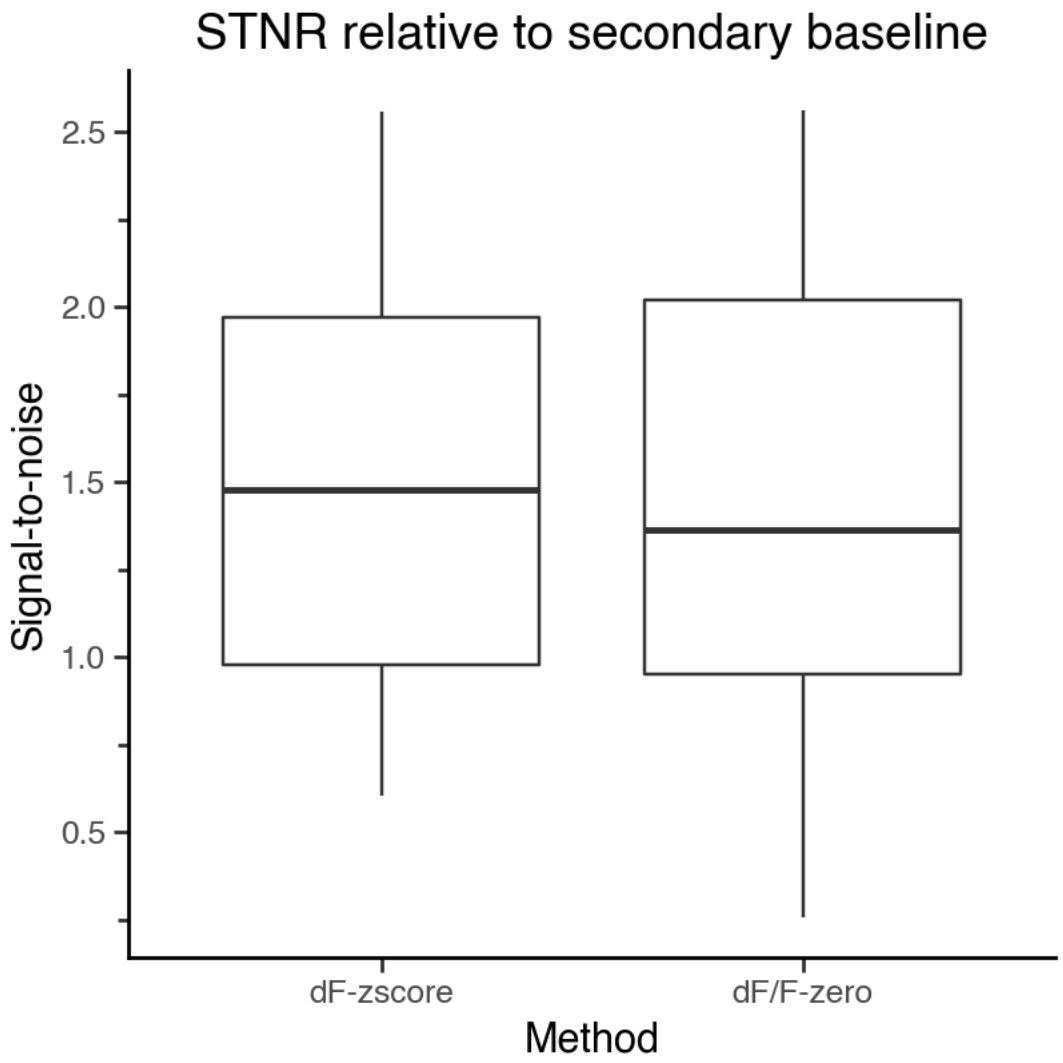


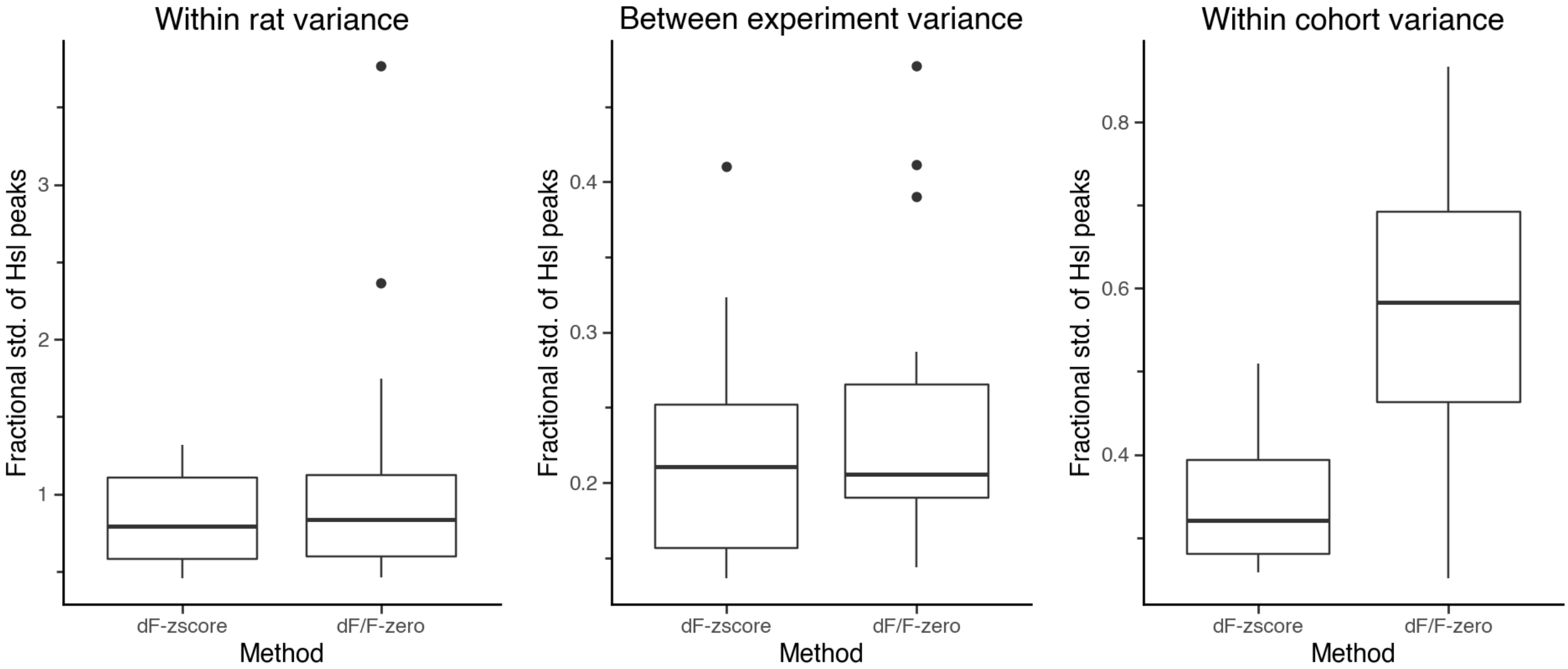
dF-zscore




Distribution of Hsl peaks
dF-zscore
dF/F-zero
Hsl Peaks (Z-score)
2
1
0
-1
A F
A M
Y F
Y M
A F
A M
Y F
Y M
Cohort (age-sex)


STNR relative to baseline
Signal-to-noise
4
3
2
1
dF-zscore
dF/F-zero
Method

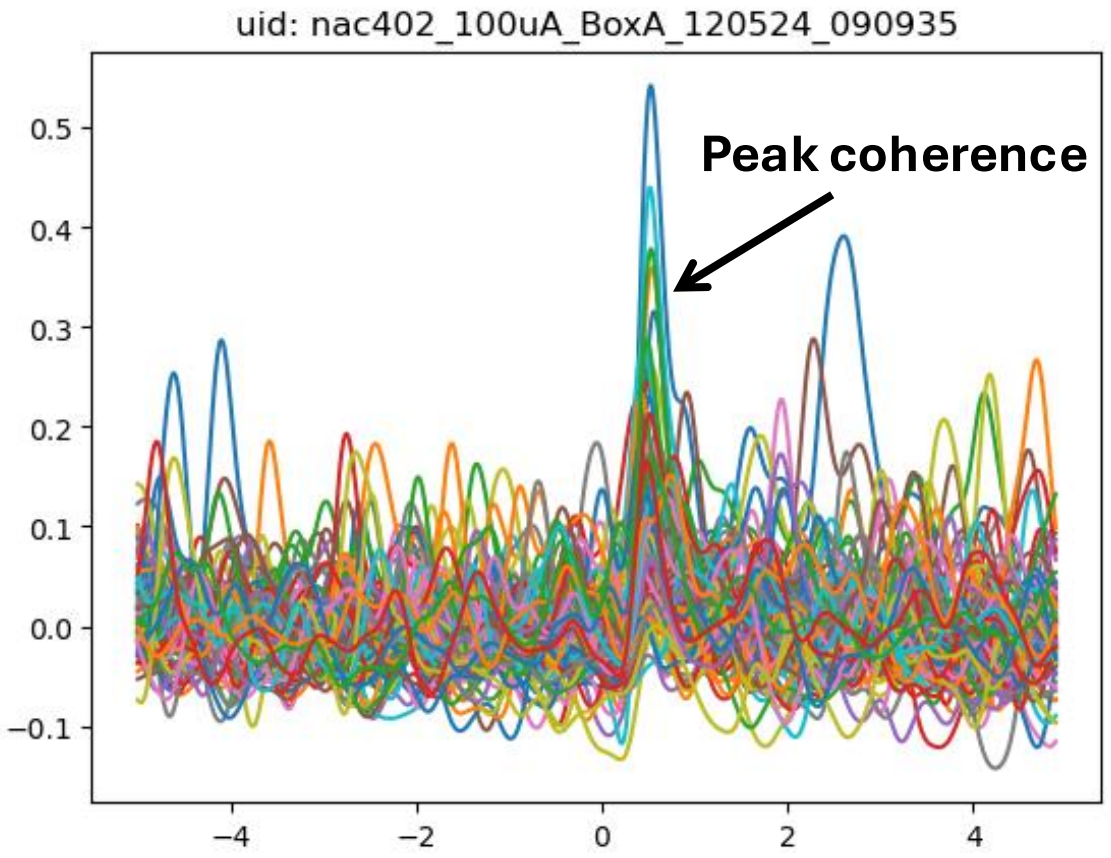

STNR relative to secondary baseline
Signal-to-noise
2.5
2.0
1.5
1.0
0.5
dF-zscore
dF/F-zero
Method



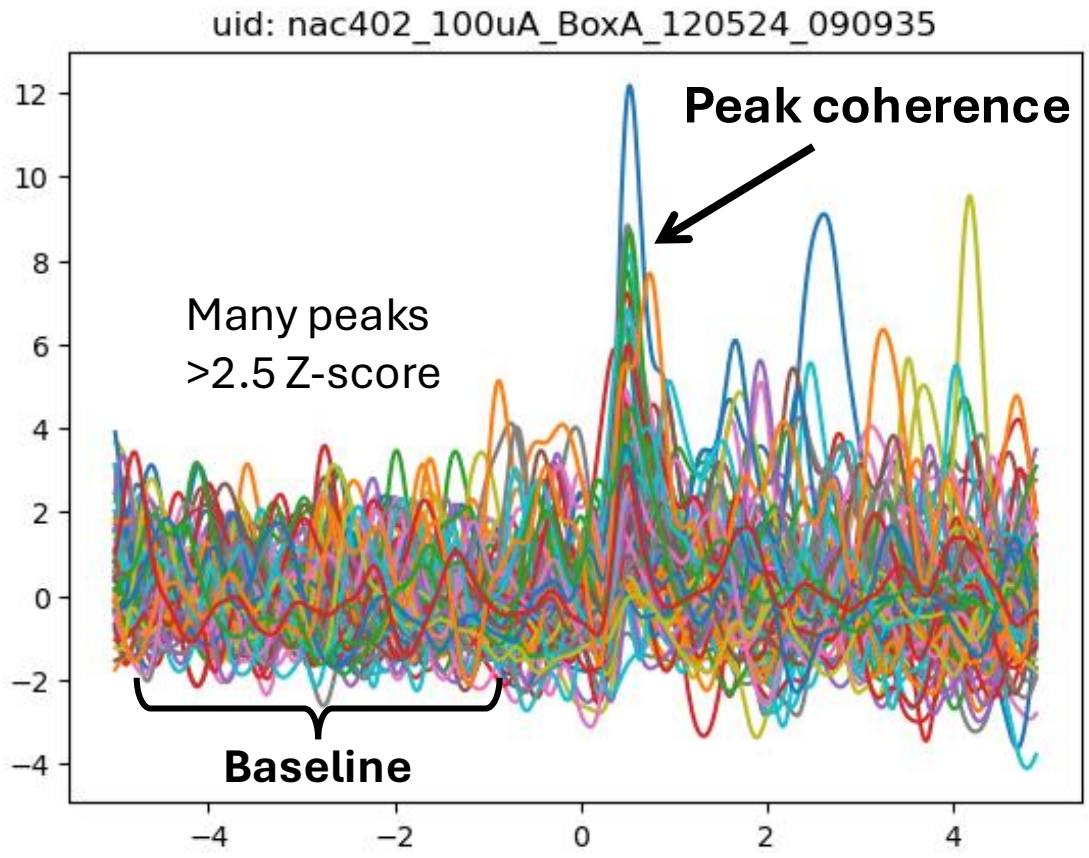
dF-zscore is better normalizing between rats and experiments, **why?**

dF/F-zero is not producing lower variance within rat, contrary to result from simulated data

dF/F-zero



dF-zscore



Hypothesis: the baseline contains random, unprompted neural dynamics (RUNDs). The Z-score normalization serves as an internal control for detector expression and dopamine activity reducing variance between rats.

pyFiberPhotometry

An object-oriented Python package to processing fiber photometry data: https://github.com/ggolde/pyFiberPhotometry.

Three core modules

1. PhotometryLoader
2. PhotometryExperiment
3. PhotometryData

Designed for bulk processing, and to be easily extendable.

Current state of the package

- Both isosbestic correction methods
- Fancy photobleaching detrending
- Many trial-wise and whole-signal normalizations

- Easy handling of large photometry data sets
- Support for simulating complex photometry data with events

- Native support TDT formats

PhotometryLoader

- A scaffold for importing data from various formats
- Subclass for custom pipelines
- Separate to make PhotometryExperiment format agnostic

```python
from pyFiberPhotometry.core.PhotometryLoader import PhotometryLoader

class CustomLoader(PhotometryLoader):
    # save params
    def __init__(self, path_to_file: str, args: dict) -> None:
        self.fpath = path_to_file
        self.args = args

    # load function
    def load(self) -> PhotometryExperiment:
        data = self.extract_data()
        obj = PhotometryExperiment(**data)
        return obj

    # actually extract data from file
    def extract_data(self):
        data = dict(
            raw_signal=None,
            raw_isosbestic=None,
            time=None,
            frequency=None,
            events=None,
            metadata=None,
        )
        return data

# usage
loader = CustomLoader('file', args={})
exp = loader.load()
```

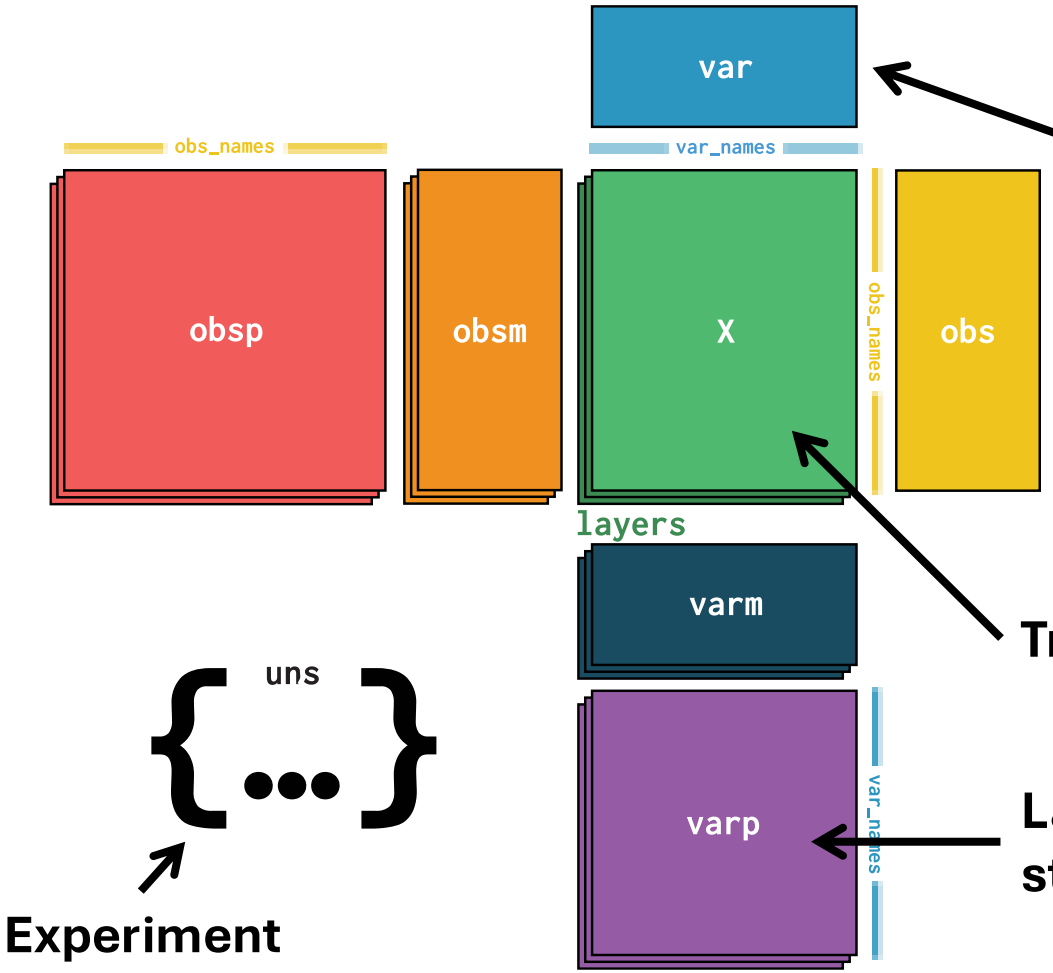
PhotometryExperiment

- Two main APIs
- Preprocess signal
- Extract trial data
- trial data stored in exp.trial_data

```
exp.preprocess_signal(
    # low pass params
    cutoff_frequency=3,
    order=4,
    # correction params
    iso_correction_method='dF/F', # dF/F, dF
    fit_using='IRLS', # OLS, IRLS, and both w/o intercepts
    maxiter=200,
    c=1.4,
    # more normalization options
    signal_normalization='none', # zscore, nullZ, none
    detrend_bleaching=False,
)
exp.extract_trial_data(
    align_to='event',
    center_on=['lever1', 'lever2'],
    trial_bounds=(-10, 10),
    event_tolerences={
        'lever1':(2, 10),
        'lever2':(2, 10),
        'loud_noise':(4, 12),
    },
    trial_normalization='none',
    event_conflict_logic='first',
)
trials = exp.trial_data
```

PhotometryData

- Stores data in anndata format




```
trials.obs # trial-wise metadata
trials.var['t'] # time points
trials.X # trial signals
trials.uns # experiment metadata
```

Usage

```
tr.obs
```

✓ 0.0s

	event	lever1	lever2	loud_noise
0	-6.653480	0.0	NaN	NaN
1	-6.607111	NaN	0.0	0.658430
2	-4.218853	0.0	NaN	NaN
3	0.000000	NaN	NaN	NaN
4	-5.204102	0.0	NaN	NaN
...	...	...	...	...
70	-5.556561	NaN	0.0	NaN
71	-3.064800	0.0	NaN	NaN
72	-4.527416	0.0	NaN	NaN
73	-4.245385	NaN	0.0	0.458836
74	0.000000	NaN	NaN	NaN

75 rows × 4 columns

```
has_l1 = ~tr.obs['lever1'].isna()
has_l2 = ~tr.obs['lever2'].isna()
has_ln = ~tr.obs['loud_noise'].isna()

tr.obs['trial_label'] = pd.NA
tr.obs.loc[has_l1, 'trial_label'] = 'type1'
tr.obs.loc[has_l2 & ~has_ln, 'trial_label'] = 'type2'
tr.obs.loc[has_l2 & has_ln, 'trial_label'] = 'type3'

tr.obs
```

✓ 0.0s

	event	lever1	lever2	loud_noise	trial_label
0	-6.653480	0.0	NaN	NaN	type1
1	-6.607111	NaN	0.0	0.658430	type3
2	-4.218853	0.0	NaN	NaN	type1
3	0.000000	NaN	NaN	NaN	<NA>
4	-5.204102	0.0	NaN	NaN	type1
...	...	...	...	...	...
70	-5.556561	NaN	0.0	NaN	type2
71	-3.064800	0.0	NaN	NaN	type1
72	-4.527416	0.0	NaN	NaN	type1
73	-4.245385	NaN	0.0	0.458836	type3
74	0.000000	NaN	NaN	NaN	<NA>

75 rows × 5 columns

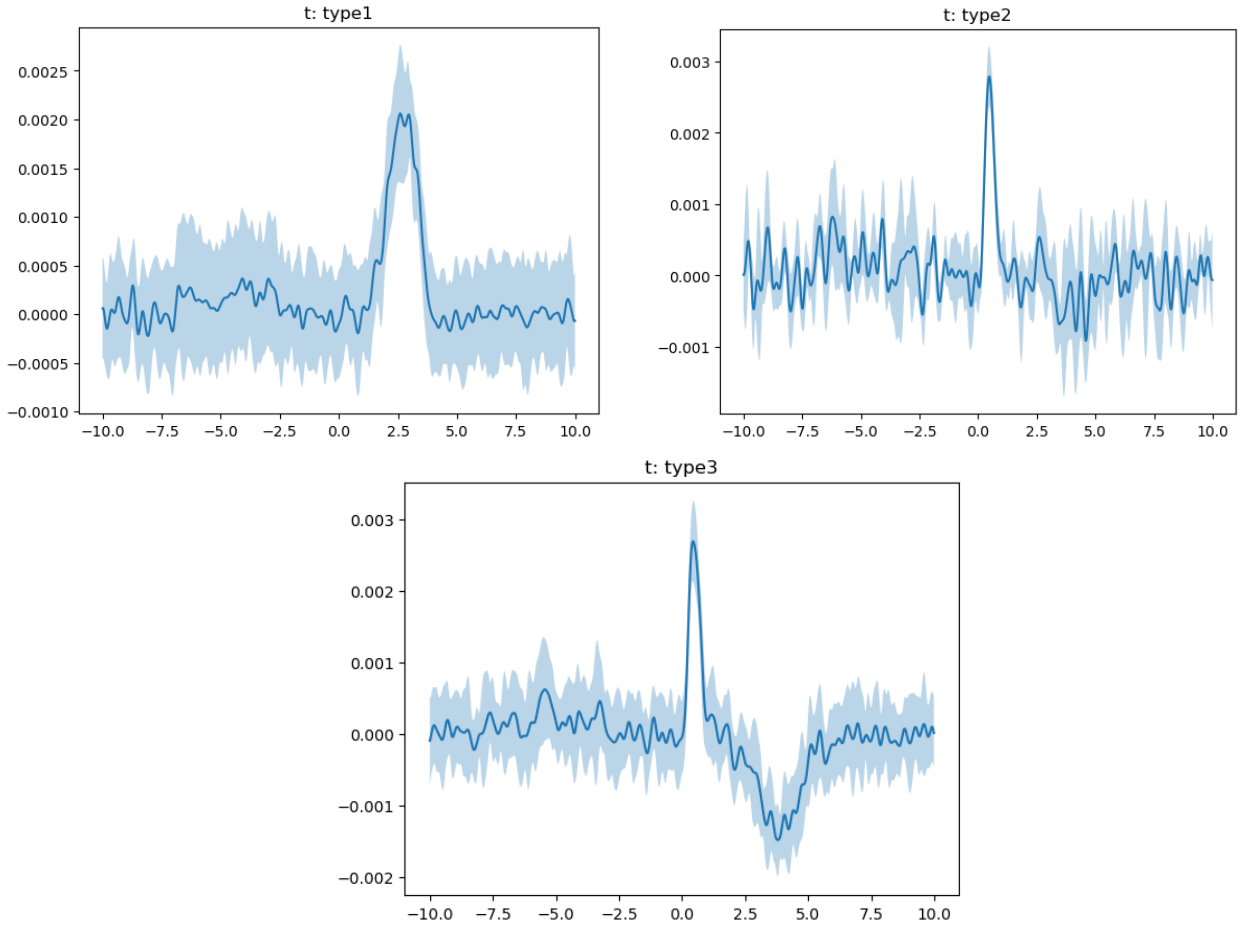
Aggregation

```
avg = tr.collapse(
    group_on=['trial_label'],
    metrics={'std' : np.std},
    data_cols=['event'],
    count_col='n_trials'
)
avg.obs
```

✓ 0.0s

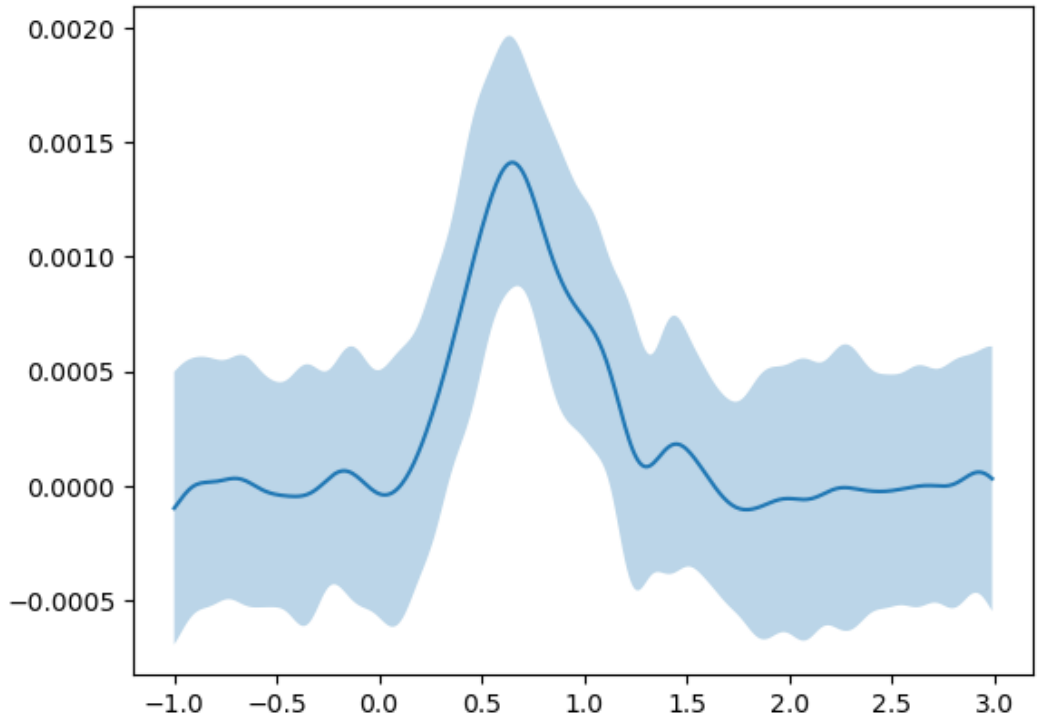
	trial_label	n_trials	event	event_std
0	type1	41	-5.256388	1.418527
1	type3	18	-5.699873	1.121605
2	type2	5	-5.364903	1.463026

```
avg.plot_groups('trial_label', err_layer='std')
```



Windowing data

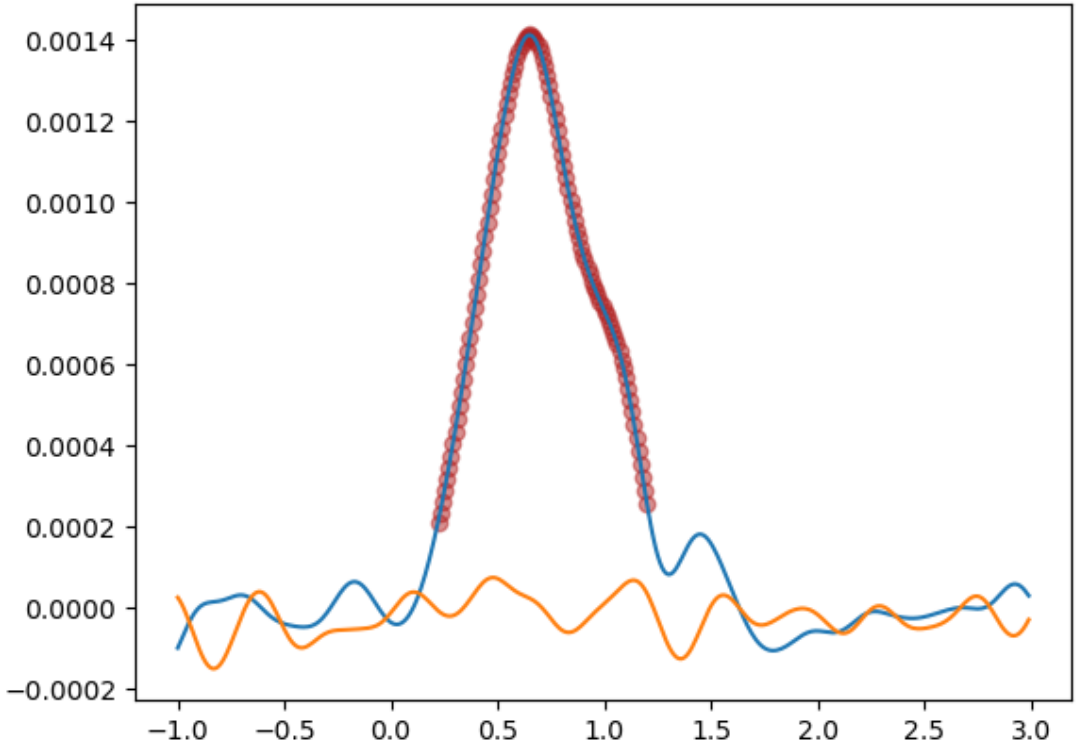
```
hsl_aligned = tr.window(
    series=tr.ts,
    centers=tr.obs['event'],
    bounds=(-1, 3),
    freq=float(tr.uns['frequency']),
    event_cols=[
        'event', 'lever1', 'lever2'
    ]
)

avg = hsl_aligned.collapse(None, metrics={'std':np.std})
avg.plot_line(0, err_layer='std')
```



Comparing Data

```python
from pyFiberPhotometry.analysis.comparison import cluster_depth_test

res = cluster_depth_test(
    X=baseline.X,
    Y=hsl_aligned.X,
    n_jobs=1,
    random_state=1,
)
```

```python
time = hsl_aligned.ts
hsl_avg = hsl_aligned.collapse(None).X[0]
noise_avg = baseline.collapse(None).X[0]

sig_idxs = np.where(res['pvals'] < 0.05)

plt.scatter(time[sig_idxs], hsl_avg[sig_idxs], c='firebrick', alpha=0.5)
plt.plot(time, hsl_avg)
plt.plot(time, noise_avg)

plt.plot()
```



Intended use

- A toolkit rather than an all-encompassing program
- Subclassing PhotometryData and/or PhotometryExperiment to create functionality customized to specific datasets
- Creation of an all-in-one “process_directory” function

Future features

- Movement artifact quantification
- More loaders for various formats
- Accessible API for cluster permutation tests
- Hyperparameter grid search

- Any other suggestions?

Alternatives

- GuPPy (https://github.com/LernerLab/GuPPy)
- PhAT (https://github.com/donaldsonlab/PhAT)
  - Both have graphical interfaces
  - Good if you do not want to do any coding

Sources

1. Keevers, L. J., & Jean-Richard-dit-Bressel, P. (2025). Obtaining artifact-corrected signals in fiber photometry via isosbestic signals, robust regression, and dF/F calculations. *Neurophotonics*, *12*(2), 025003. https://doi.org/10.1117/1.NPh.12.2.025003